

Module 2

3D Transformations and viewing

3D Transformations

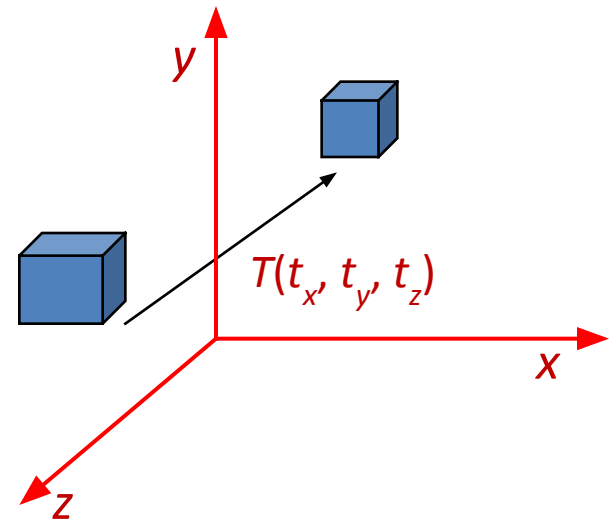
- 3-D Transformation is the process of manipulating the view of a three-D object with respect to its original position by modifying its physical attributes through various methods of transformation like Translation, Scaling, Rotation, Shear, etc.
- **Properties of 3-D Transformation:**
 - Lines are preserved,
 - Parallelism is preserved,
 - Proportional distances are preserved.

TRANSLATION

- In a three-dimensional homogeneous coordinate representation, a point is translated from position $P = (x, y, z)$ to position $P' = (x', y', z')$ with the matrix operation

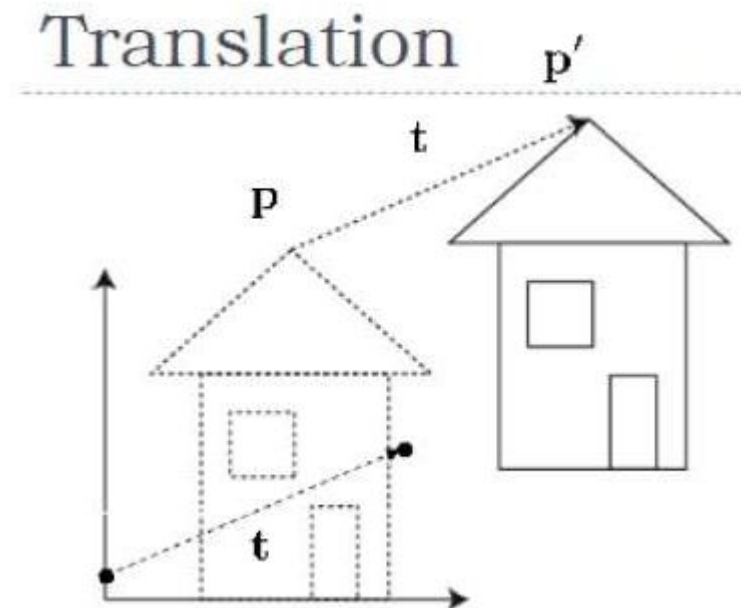
$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$P' = T(t_x, t_y, t_z)P$$



- If P is a point having co-ordinates in three directions (x, y, z) is translated, then after translation its coordinates will be (x' y¹ z¹) after translation. T_x T_y T_z are translation vectors in x, y, and z directions respectively.

$$\begin{aligned}x^1 &= x + T_x \\ y^1 &= y + T_y \\ z^1 &= z + T_z\end{aligned}$$



Example

- A point has coordinates in the x, y, z direction i.e., (5, 6, 7). The translation is done in the x-direction by 3 coordinate and y direction. Three coordinates and in the z- direction by two coordinates. Shift the object. Find coordinates of the new position.

- Co-ordinate of the point are (5, 6, 7)

Translation vector in x direction = 3

Translation vector in y direction = 3

Translation vector in z direction = 2

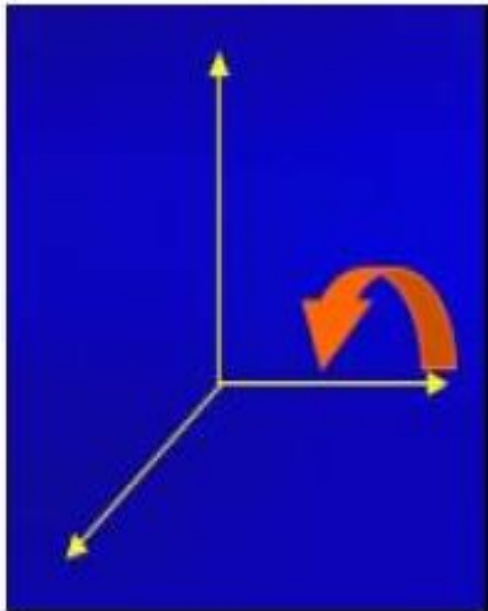
x becomes $x^1=8$

y becomes $y^1=9$

z becomes $z^1=9$

ROTATION

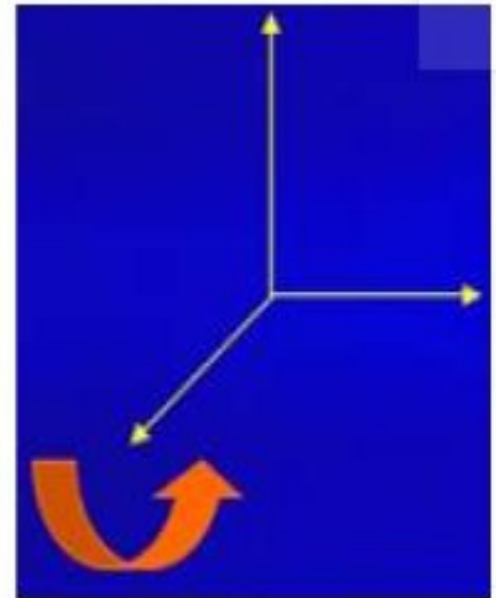
- It is moving of an object about an angle. Movement can be anticlockwise or clockwise.
- 3D rotation is complex as compared to the 2D rotation.
- For 2D we describe the angle of rotation, but for a 3D angle of rotation and axis of rotation are required.
- The axis can be either x or y or z.
- Positive rotation angles produce counterclockwise rotations about a coordinate axis, if looking along the positive half of the axis toward the coordinate origin, and negative rotation angles produce clockwise rotations.



Rotation about x-axis



Rotation about y-axis



Rotation about z-axis

z-axis rotation

- A Z-axis rotation in 3D involves rotating a point around the Z-axis by a specified angle, θ . This transformation keeps the Z-coordinate unchanged while modifying the X and Y coordinates.

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z$$

- **Parameter θ specifies the rotation angle. In homogeneous coordinate form, the three-dimensional z-axis rotation equations are expressed as**

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$P' = R_z(\theta)P$$

x-axis rotation

- An X-axis rotation in 3D involves rotating a point around the X-axis by a specified angle, θ . This transformation keeps the X-coordinate unchanged while modifying the Y and Z coordinates.
- Given a point (x, y, z) , the new coordinates (x', y', z') after rotation by angle θ around the X-axis can be calculated as follows:

$$\begin{aligned} y' &= y \cos \theta - z \sin \theta \\ z' &= y \sin \theta + z \cos \theta \\ x' &= x \end{aligned}$$

The transformation can be represented by a rotation matrix:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$P' = R_x(\theta)P$$

y-axis rotation

- A Y-axis rotation in 3D involves rotating a point around the Y-axis by a specified angle, θ .
- This transformation keeps the Y-coordinate unchanged while modifying the X and Z coordinates.
- Given a point (x, y, z) , the new coordinates (x', y', z') after rotation by angle θ around the Y-axis can be calculated as follows:

$$z' = z \cos \theta - x \sin \theta$$

$$x' = z \sin \theta + x \cos \theta$$

$$y' = y$$

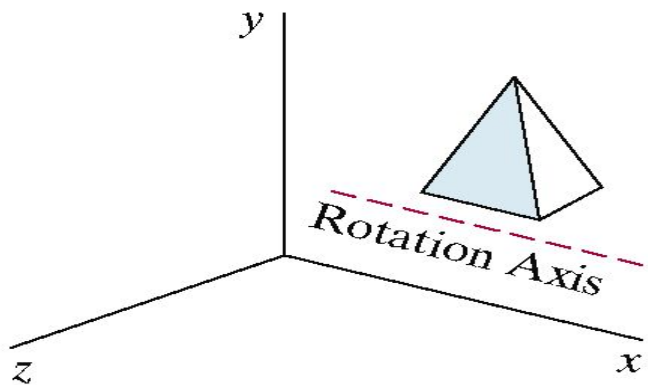
The transformation can be represented by a rotation matrix:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$P' = R_y(\theta)P$$

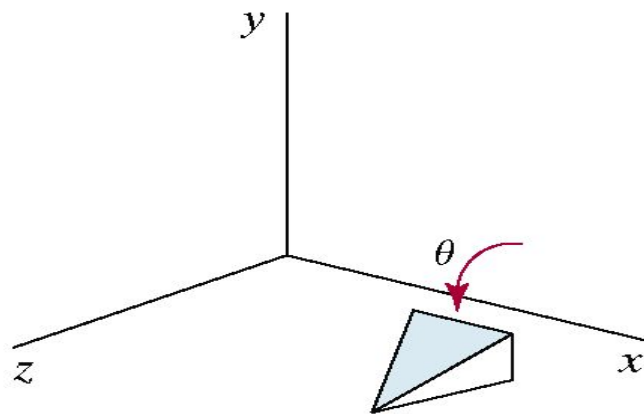
Rotated about an axis that is parallel to one of the coordinate axis

1. Translate the object so that the rotation axis coincides with the parallel coordinate axis.
2. Perform the specified rotation about that axis.
3. Translate the object so that the rotation axis is moved back to its original position.



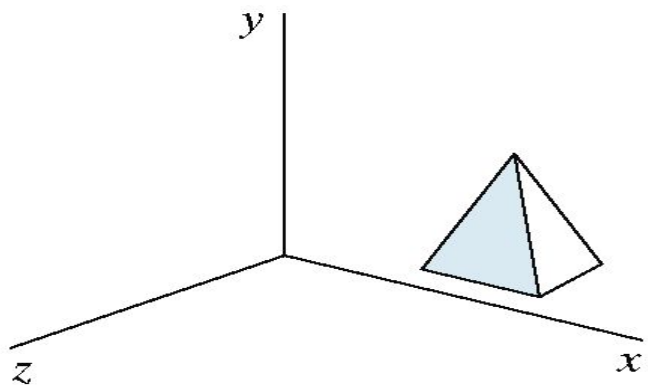
(a)

Original Position of Object



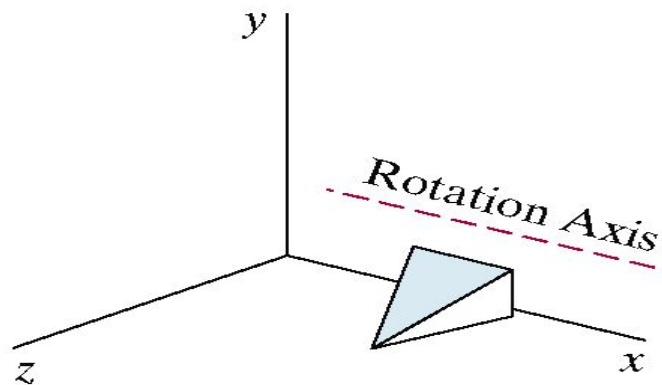
(c)

Rotate Object Through Angle θ



(b)

Translate Rotation Axis onto x Axis



(d)

Translate Rotation Axis to Original Position

- Any coordinate position **P** on the object is transformed with the sequence shown as

$$\mathbf{P}' = \mathbf{R}(\theta) \bullet \mathbf{P}$$

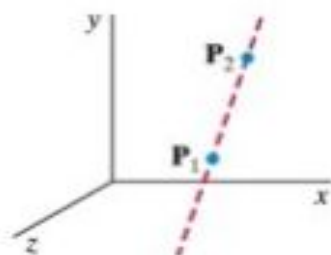
Where the composite matrix for the transformation is

$$= \mathbf{T}^{-1} \bullet \mathbf{R}_x(\theta) \bullet \mathbf{T} \bullet \mathbf{P}$$

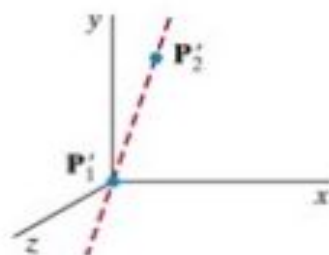
$$(\text{where } \mathbf{R}(\theta) = \mathbf{T}^{-1} \bullet \mathbf{R}_x(\theta) \bullet \mathbf{T})$$

Rotated about an axis that is not parallel to one of the coordinate axis(Rotated about an arbitrary line)

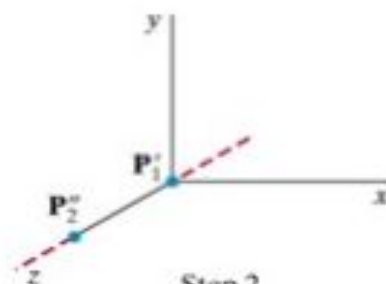
- 1 Translate the object so that the rotation axis pass through the coordinate origin.
2. Rotate the object so that the axis of rotation coincides with one of the coordinate axes.
3. Perform the specified rotation about that coordinate axis.
4. Apply inverse rotations to bring the rotation axis back to its original orientation.
5. Apply the inverse translation to bring the rotation axis back to its original position



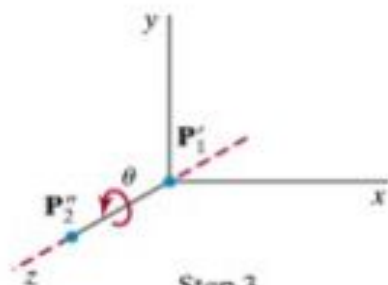
Initial
Position



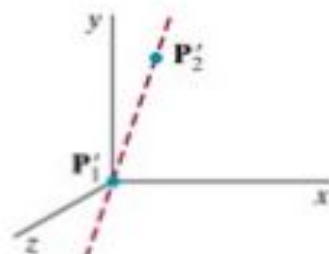
Step 1
Translate
 P_1 to the Origin



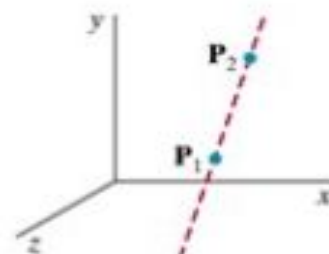
Step 2
Rotate P'_2
onto the z Axis



Step 3
Rotate the
Object Around the
z Axis



Step 4
Rotate the Axis
to its Original
Orientation



Step 5
Translate the
Rotation Axis
to its Original
Position

- A rotation axis can be defined with two coordinate positions or with one coordinate point and direction angles (or direction cosines) between the rotation axis and two of the coordinate axes
- We assume that the rotation axis is defined by two points, as illustrated in the initial position , and that the direction of rotation is to be counterclockwise when looking along the axis from
- An axis vector is then defined by the two points as

$$\mathbf{v} = \mathbf{P2} - \mathbf{P1}$$

$$= (x2 - x1, y2 - y1, z2 - z1)$$

- A unit vector u is then defined along the rotation axis as

$$\mathbf{u} = \frac{\mathbf{V}}{|\mathbf{V}|} = a, b, c$$

- where the components a , b , and c of unit vector u are the direction cosines for the rotation **axis**:

$$a = \frac{x_2 - x_1}{|\mathbf{V}|}, \quad b = \frac{y_2 - y_1}{|\mathbf{V}|}, \quad c = \frac{z_2 - z_1}{|\mathbf{V}|}$$

- The first step in the transformation sequence for the desired rotation is to set up the translation matrix that repositions the rotation axis so that it passes through the coordinate origin.

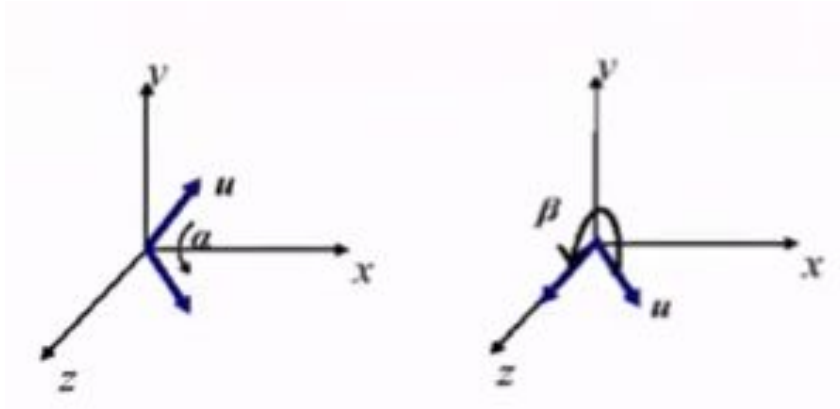
- Translation matrix is

$$T = \begin{bmatrix} 1 & 0 & 0 & -x_1 \\ 0 & 1 & 0 & -y_1 \\ 0 & 0 & 1 & -z_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Next we formulate the transformations that will put the rotation axis on the **z** axis.
- We can use the coordinate-axis rotations to accomplish this alignment in two steps.
- There are a number of ways to perform the two steps.

Align **u** with the z axis

- Rotate around x axis to get u into the xz plane
- Rotate around y axis to get u aligned with the z axis



- Rotate around x axis to get u into the xz plane

$$\sin \alpha = \frac{b}{\sqrt{b^2 + c^2}} = \frac{b}{d}$$

$$\cos \alpha = \frac{c}{\sqrt{b^2 + c^2}} = \frac{c}{d}$$

$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c/d & -b/d & 0 \\ 0 & b/d & c/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

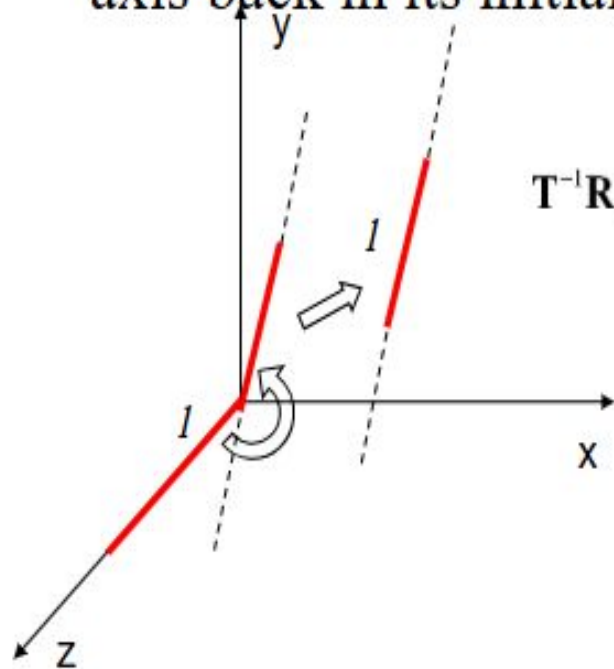
- Rotate around y axis to get u aligned with the z axis

$$\begin{aligned} \sin \beta &= \frac{a}{l}, & \cos \beta &= \frac{d}{l} \\ l^2 &= a^2 + b^2 + c^2 = a^2 + d^2 \\ d &= \sqrt{b^2 + c^2} \end{aligned} \quad \mathbf{R}_y(\beta) = \begin{bmatrix} \cos \beta & 0 & -\sin \beta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} d/l & 0 & -a/l & 0 \\ 0 & 1 & 0 & 0 \\ a/l & 0 & d/l & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Rotate about z axis by the desired angle.

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Step 5. Apply the reverse transformation to place the axis back in its initial position



$$\mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta) = \begin{bmatrix} 1 & 0 & 0 & -x_1 \\ 0 & 1 & 0 & -y_1 \\ 0 & 0 & 1 & -z_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & \sin \alpha & 0 \\ 0 & -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

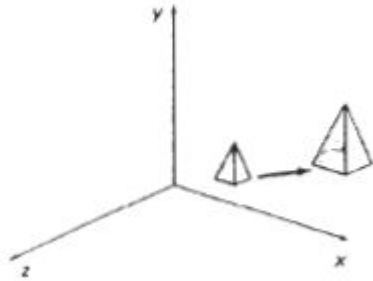
$$\begin{bmatrix} \cos \beta & 0 & \sin \beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}(\theta) = \mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta)\mathbf{R}_z(\theta)\mathbf{R}_y(\beta)\mathbf{R}_x(\alpha)\mathbf{T}$$

Scaling(about origin)

- Scaling is a method used to change the size of an object.
- It can be used to increase or decrease the object size.
- In scaling, the object can be altered in any of the x, y, z directions through S_x , S_y , S_z scaling factors.
- If scaling factor > 1 , then the object size is increased.
- If scaling factor < 1 , then the object size is reduced.

- The matrix expression for the scaling transformation of a position $P = (\mathbf{x}, y, z)$ relative to the coordinate origin can be written as



$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$P' = S(s_x, s_y, s_z)P$$

- Explicit expressions for the coordinate transformations for scaling relative to the origin are

$$\mathbf{x}' = \mathbf{x} \cdot s$$

$$y' = y \cdot s$$

$$z' = z \cdot s$$

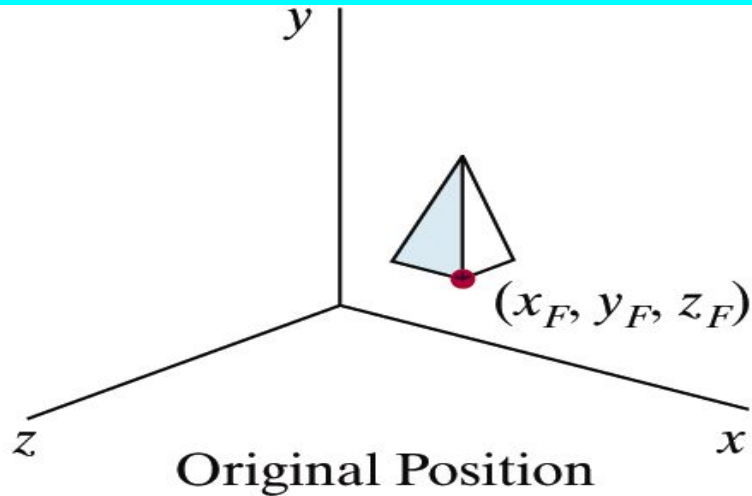
- Scaling with respect to a selected fixed position (x, y, z) can be represented with the following transformation sequence:

1. Translate the fixed point to the origin.
2. Scale the object relative to the coordinate origin.
3. Translate the fixed point back to its original position.

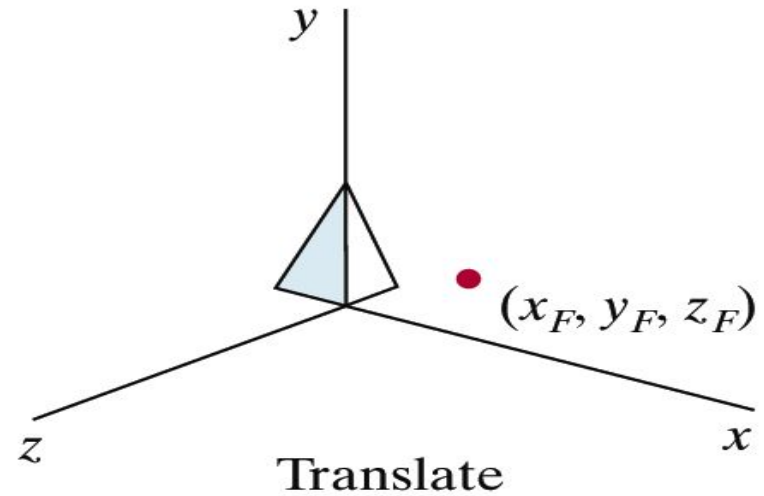
In matrix form: $T(P_x, P_y, P_z) (S_x, S_y, S_z) T(-P_x, -P_y, -P_z) * P$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & P_x \\ 0 & 1 & 0 & P_y \\ 0 & 0 & 1 & P_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -P_x \\ 0 & 1 & 0 & -P_y \\ 0 & 0 & 1 & -P_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

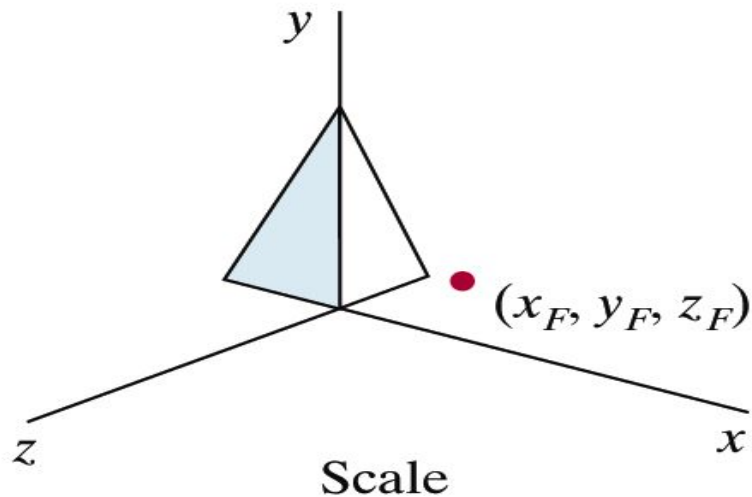
3D Scaling w.r.t to a fixed point



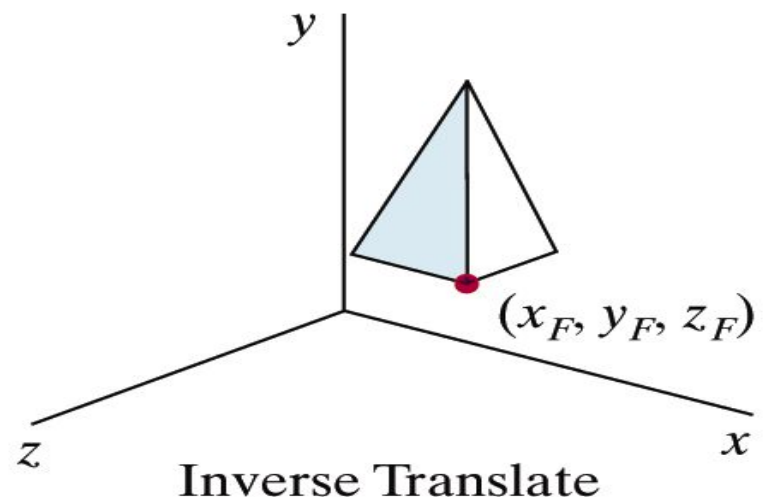
(a)



(b)



(c)



(d)

$$\mathbf{T} \cdot \mathbf{S} \cdot \mathbf{T}^{-1} = \begin{bmatrix} S_x & 0 & 0 & (1 - S_x) x_f \\ 0 & S_y & 0 & (1 - S_y) y_f \\ 0 & 0 & S_z & (1 - S_z) z_f \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

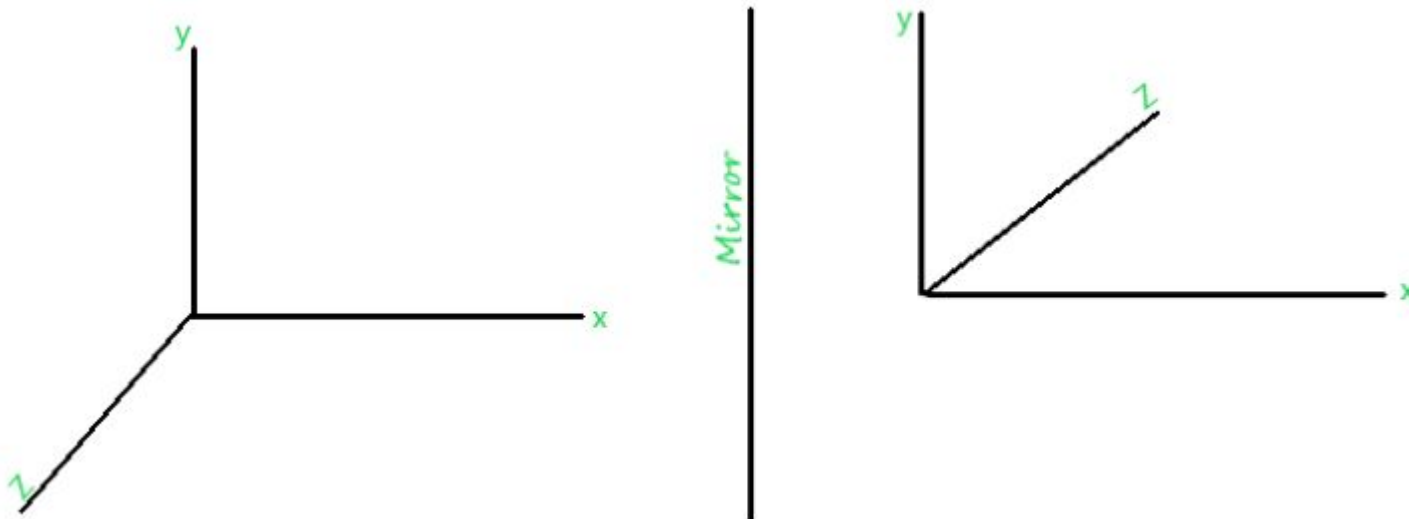
Reflection

- A transformation that produces a mirror image of an object.
- Reflections relative to a given axis are equivalent to 180 degree rotations about that axis.
- . The reflected object is always formed on the other side of mirror.
- . The size of reflected object is same as the size of original object
- Three kinds of Reflections are possible in 3D space:
 - Reflection along the X-Y plane.
 - Reflection along Y-Z plane.
 - Reflection along X-Z plane.

Reflection along the X-Y plane

- The Reflection transformation matrix is used to perform the reflection operation over the 3D image, which is as follows:

$$R_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

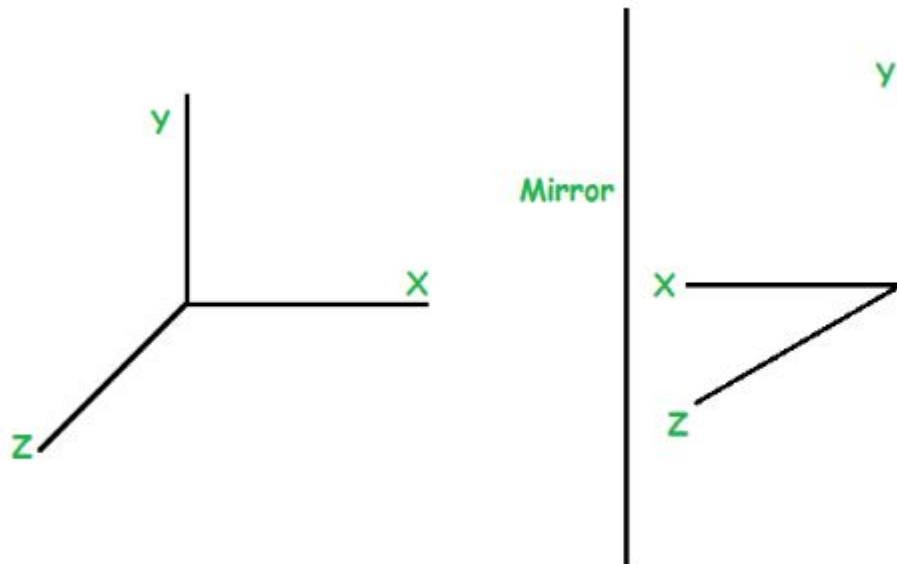


Reflection along x-y plane

Reflection along the Y-Z plane

- The reflection transformation matrix for y-z axes is as follows:*

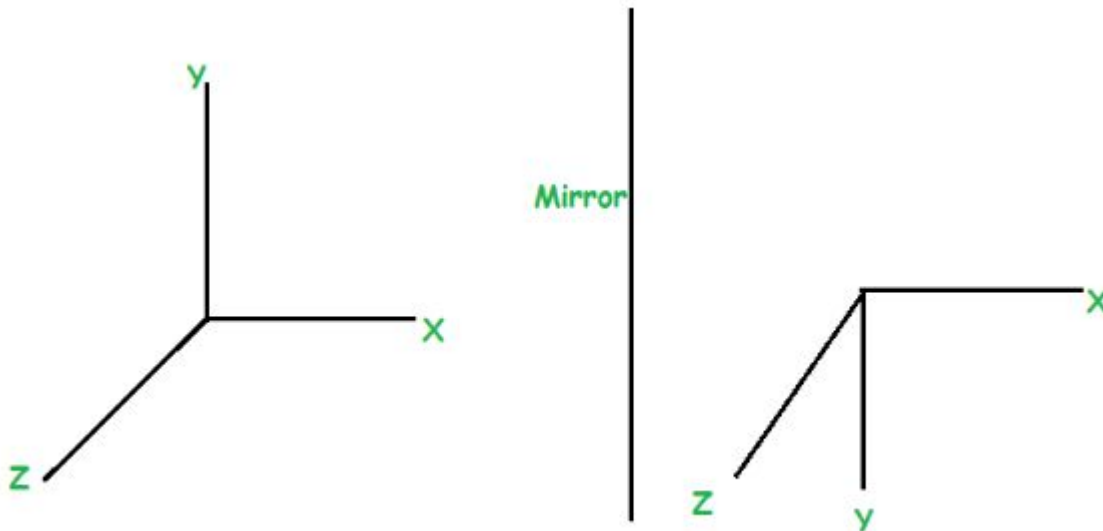
$$R_{yz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Reflection along the X-Z plane

- The Reflection transformation matrix for z-x axes is as follows:

$$R_{zx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



- Given a 3D triangle with coordinate points $A(3, 4, 1)$, $B(6, 4, 2)$, $C(5, 6, 3)$. Apply the reflection on the XZ plane and find out the new coordinates of the object.

- Applying the reflection equations, we have-

$$X_{\text{new}} = X_{\text{old}} = 5$$

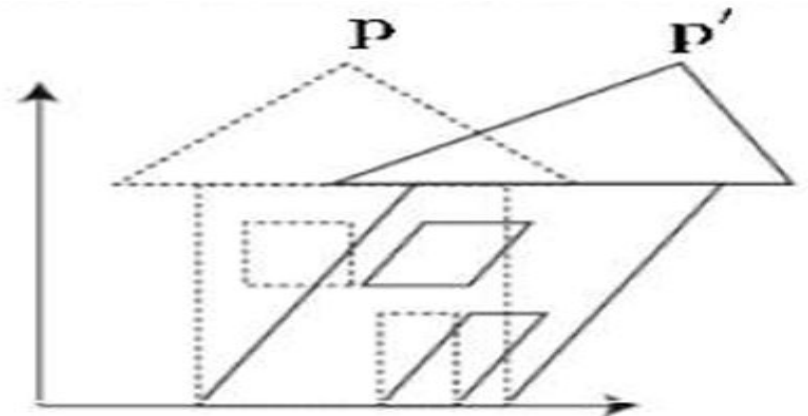
$$Y_{\text{new}} = -Y_{\text{old}} = -6$$

$$Z_{\text{new}} = Z_{\text{old}} = 3$$

- Thus, New coordinates of the triangle after reflection = A (3, -4, 1), B(6, -4, 2), C(5, -6, 3).

Shearing

- Shearing is the process of slanting an object in 3D space either in x, y, or in the z-direction.
- Shearing changes(or deformed) the shape of the object.
- The types of shearing transformation.
 - Shearing in X-direction.
 - Shearing in y-direction
 - Shearing in z-direction.



Shearing in X-Direction

- The coordinate of X remains unchanged while the coordinate of Y and Z is changed.

$$X_{\text{new}} = X_{\text{old}}$$

$$Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}}$$

$$Z_{\text{new}} = Z_{\text{old}} + Sh_z \times X_{\text{old}}$$

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ Sh_y & 1 & 0 & 0 \\ Sh_z & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

Shearing in Y-Direction

- The coordinate of Y remains unchanged while the coordinate of X and Z are changed.
- Shearing in Y axis is achieved by using the following shearing equations-

$$X_{\text{new}} = X_{\text{old}} + Sh_x \times Y_{\text{old}}$$

$$Y_{\text{new}} = Y_{\text{old}}$$

$$Z_{\text{new}} = Z_{\text{old}} + Sh_z \times Y_{\text{old}}$$

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & Sh_z & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

Shearing in Z-Direction

- The coordinate of Z remains unchanged while the coordinate of X and Y are changed.
- Shearing in Z axis is achieved by using the following shearing equations-

$$X_{\text{new}} = X_{\text{old}} + Sh_x \times Z_{\text{old}}$$

$$Y_{\text{new}} = Y_{\text{old}} + Sh_y \times Z_{\text{old}}$$

$$Z_{\text{new}} = Z_{\text{old}}$$

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & Sh_x & 0 \\ 0 & 1 & Sh_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

- Given a 3D triangle with points (0, 0, 0), (1, 1, 2) and (1, 1, 3). Apply shear parameter 2 on X axis, 2 on Y axis and 3 on Z axis and find out the new coordinates of the object.

For Coordinates A(0, 0, 0)

- $X_{\text{new}} = X_{\text{old}} = 0$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 0 + 2 \times 0 = 0$
- $Z_{\text{new}} = Z_{\text{old}} + Sh_z \times X_{\text{old}} = 0 + 3 \times 0 = 0$

For Coordinates B(1, 1, 2)

- $X_{\text{new}} = X_{\text{old}} = 1$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 1 + 2 \times 1 = 3$
- $Z_{\text{new}} = Z_{\text{old}} + Sh_z \times X_{\text{old}} = 2 + 3 \times 1 = 5$

For Coordinates C(1, 1, 3)

- $X_{\text{new}} = X_{\text{old}} = 1$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 1 + 2 \times 1 = 3$
- $Z_{\text{new}} = Z_{\text{old}} + Sh_z \times X_{\text{old}} = 3 + 3 \times 1 = 6$

COMPOSITE TRANSFORMATIONS

- A number of transformations or sequence of transformations can be combined into single one called as composition.
- This concatenation is carried out from right to left, where the rightmost matrix is the first transformation to be applied to an object and the leftmost matrix is the last transformation.

- Row matrix

- Left -1st transformation

- Right-last transformation

$$[x', y', z'] = [x, y, z] \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Column matrix

- Left -Last transformation

- Right-1st transformation

$$[x', y', z'] = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Two successive translations.

- When you apply two translations, T1 and T2, one after the other, the resulting transformation is equivalent to a single translation, T3.

Mathematically, this can be represented as:

$$T3 = T2 * T1$$

- To find T3, we multiply the matrices T1 and T2.
Let's assume:

T1 translates by (tx1, ty1, tz1)

T2 translates by (tx2, ty2, tz2)

$$T1 = \begin{bmatrix} 1 & 0 & 0 & tx1 \\ 0 & 1 & 0 & ty1 \\ 0 & 0 & 1 & tz1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T2 = \begin{bmatrix} 1 & 0 & 0 & tx2 \\ 0 & 1 & 0 & ty2 \\ 0 & 0 & 1 & tz2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T3 = \begin{bmatrix} 1 & 0 & 0 & tx1+tx2 \\ 0 & 1 & 0 & ty1+ty2 \\ 0 & 0 & 1 & tz1+tz2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Two Successive Scaling

- When you apply two scaling operations, S1 and S2, one after the other, the resulting transformation is equivalent to a single scaling operation, S3.
- Mathematically, this can be represented as:
$$S3 = S2 * S1$$
- S1 scales by factors (Sx1, Sy1, Sz1) S2 scales by factors (Sx2, Sy2, Sz2)

$$S1 = \begin{bmatrix} Sx1 & 0 & 0 & 0 \\ 0 & Sy1 & 0 & 0 \\ 0 & 0 & Sz1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$S2 = \begin{bmatrix} Sx2 & 0 & 0 & 0 \\ 0 & Sy2 & 0 & 0 \\ 0 & 0 & Sz2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiplying these matrices, we get:

$$S3 = \begin{bmatrix} Sx1*Sx2 & 0 & 0 & 0 \\ 0 & Sy1*Sy2 & 0 & 0 \\ 0 & 0 & Sz1*Sz2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Two successive rotations about any one of the coordinate axis

- When you perform two rotations, R1 and R2, one after the other, the resulting transformation is equivalent to a single rotation, R3.
- Mathematically, this can be represented as:

$$R3 = R2 * R1$$

- **Two Rotations About the Z-Axis**

Let's consider two rotations about the z-axis:

- **Rotation R1:** Rotate by an angle θ_1 .
- **Rotation R2:** Rotate by an angle θ_2 .

- The rotation matrices for R1 and R2 are:

$$R1 = \begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) & 0 & 0 \\ \sin(\theta_1) & \cos(\theta_1) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R2 = \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 & 0 \\ \sin(\theta_2) & \cos(\theta_2) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- $R3 = \begin{bmatrix} \cos(\theta_1)\cos(\theta_2) - \sin(\theta_1)\sin(\theta_2) & -\sin(\theta_1)\cos(\theta_2) - \cos(\theta_1)\sin(\theta_2) & 0 & 0 \\ \sin(\theta_1)\cos(\theta_2) + \cos(\theta_1)\sin(\theta_2) & \cos(\theta_1)\cos(\theta_2) - \sin(\theta_1)\sin(\theta_2) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

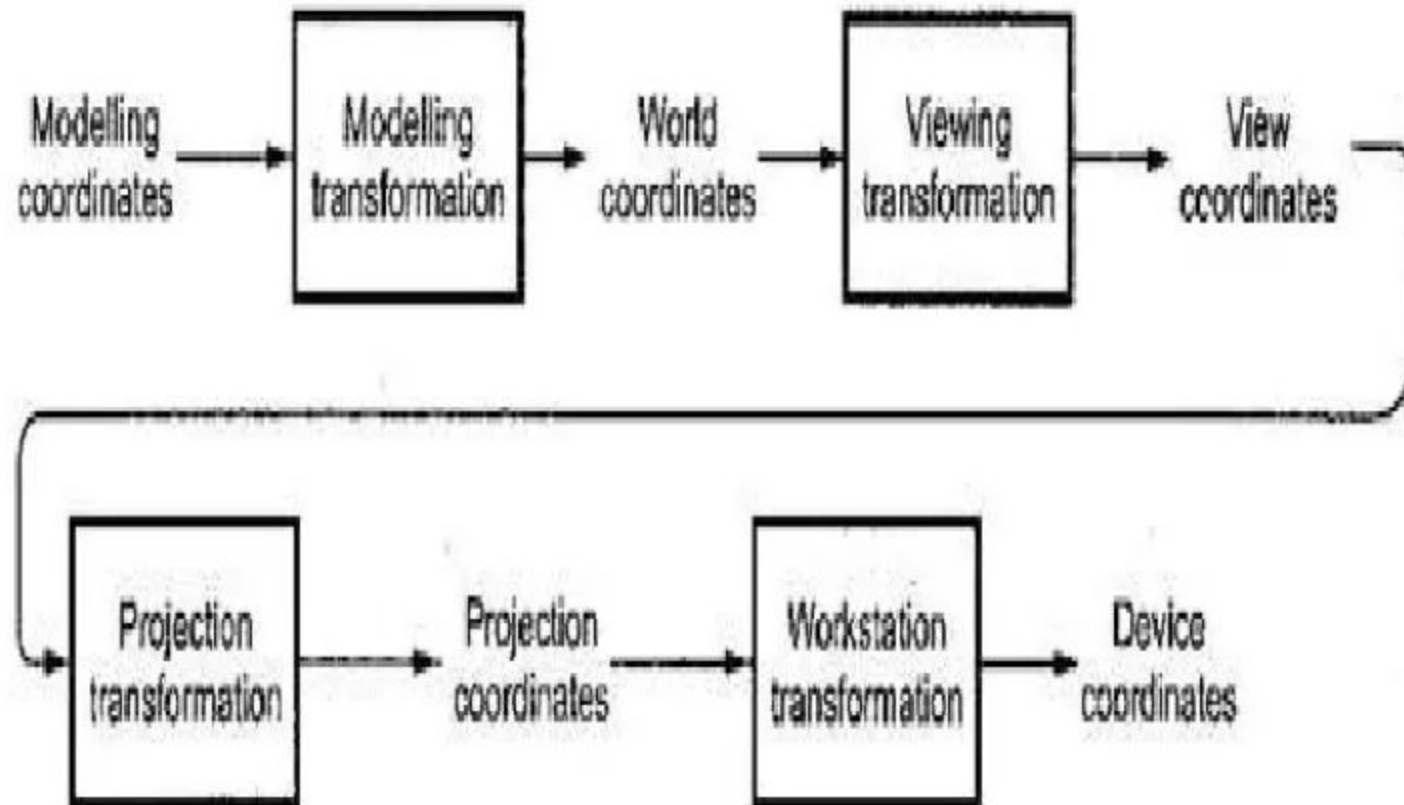
Why Composite Transformations are Important

- **Complex Modeling:** By combining multiple transformations, you can create intricate shapes and models that would be difficult to define with a single transformation.
- **Animation:** Composite transformations are crucial for creating smooth and realistic animations. For example, to animate a character walking, you might combine translations and rotations for different body parts.
- **3D Rendering:** In 3D rendering, composite transformations are used to position and orient objects in 3D space, apply lighting and shading, and project the 3D scene onto a 2D screen.
- **Game Development:** Games heavily rely on composite transformations to create dynamic environments, character movement, and object interactions.

Modelling and coordinate transformation

- Modelling transformation functions can be applied to create hierarchical representation for three-dimensional objects.
- We can define three-dimensional object shapes in local (modeling) coordinates, then we construct a scene or a hierarchical representation with instances of the individual objects.
- That is, we transform object descriptions from modeling coordinates to world coordinates or to another system in the hierarchy.

VIEWING PIPELINE



The coordinates in which we create individual objects (models) are model (or object) coordinates.

When we assemble several objects into a scene, we can describe them by world coordinates.

After transformation into the coordinate system of the camera (viewer), then they become viewing coordinates.

Also, Their projection onto a common plane (window) yields device-independent normalized coordinates.

Finally, after mapping those normalized coordinates to a specific device, we get device coordinates.

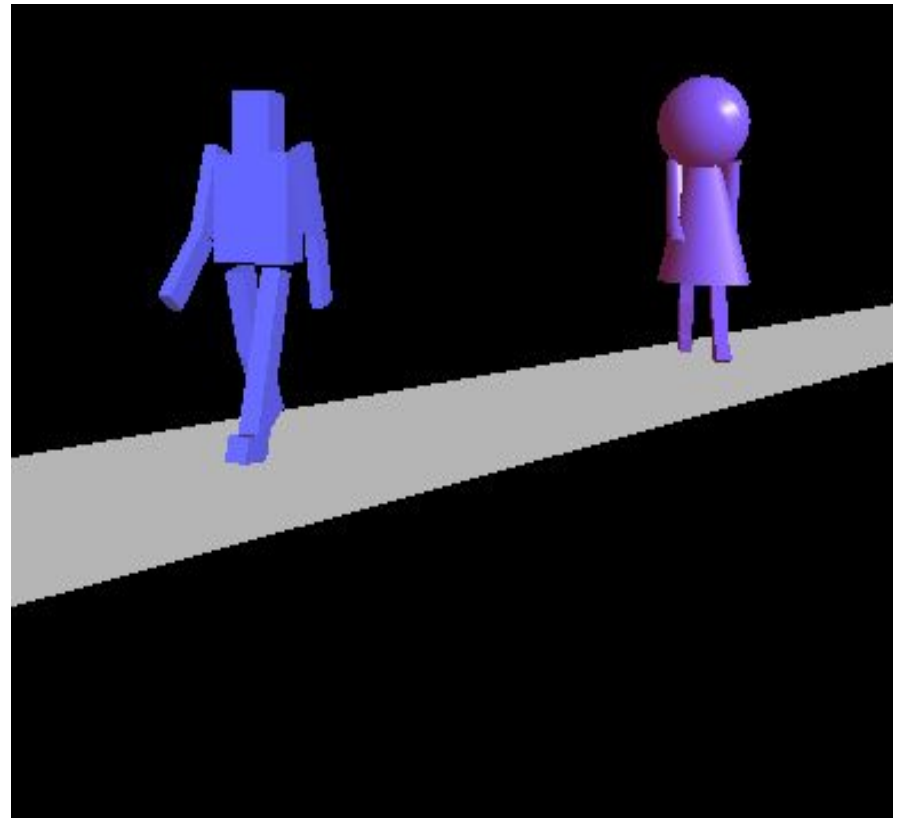
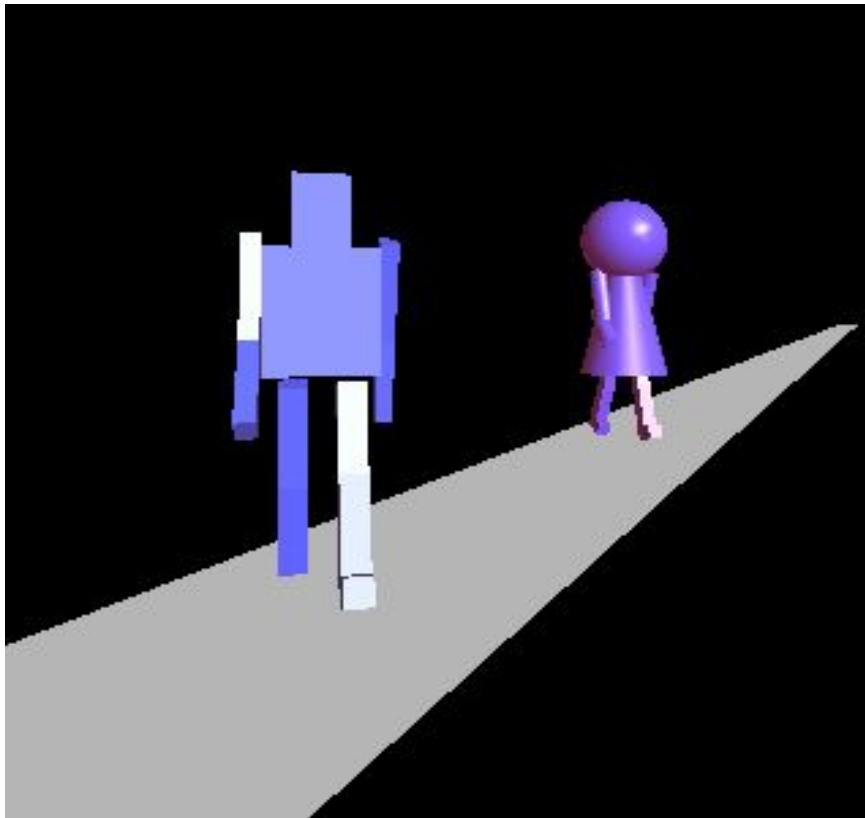
- **1. Modeling Coordinates:** This is where the 3D objects are defined in their own coordinate system (often called model space).
- **2. Modeling Transformation:** This step transforms the objects from their model space into a common world coordinate system.
- **3. World Coordinates:** This represents the common world coordinate system where all objects are now positioned.
- **4. Viewing Transformation:** This step defines the perspective of the viewer.

- **5. Viewing Coordinates:** This represents the coordinate system relative to the camera.
- **6. Projection Transformation:** This step projects the 3D objects onto a 2D viewing plane, effectively creating the illusion of depth on a flat screen.
- **7. Projection Coordinates:** This represents the coordinates of the projected objects on the viewing plane.
- **8. Workstation Transformation:** This step transforms the projected image to match the specific display device's resolution and aspect ratio.
- **9. Device Coordinates:** This represents the final coordinates of the image on the display device.

VIEWING COORDINATES

View

- pictures for observing objects with different ways are **views**.
- View results are related with the shape and size of scene objects, the position and direction of viewpoint.



Classical Viewing

- Viewing requires three basic elements
 - One or more **objects**
 - A **viewer** with a projection surface
 - **Projectors** that go from the object(s) to the projection surface
- Classical views are based on the **relationship** among these elements
 - The viewer picks up the object and orients it
- Each object is assumed to be constructed from **flat principal faces**
 - Buildings, polyhedra, manufactured objects

3D VIEWING PARAMETERS

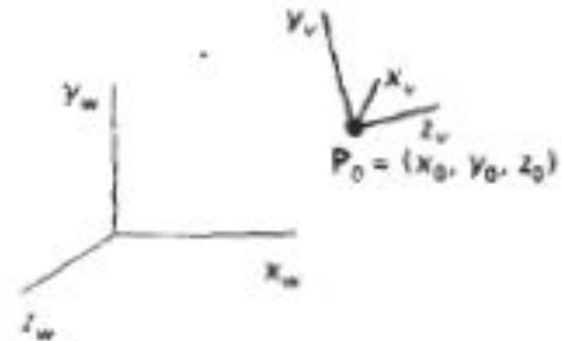
- View
- View plane
- Normal vector
- View-Up Vector
- Viewing distance

View

- A view of an object in 3D is similar to photographing the object.
- Whatever appears in the viewfinder is projected into the flat film surface.
- The type and the size of the camera lens determines which parts of the scene appear in the final picture.

Specifying the View Plane

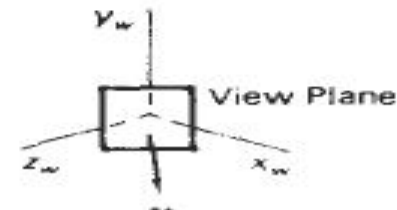
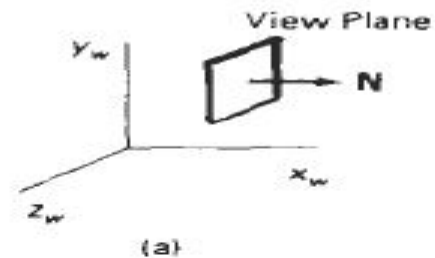
- We choose a particular view for a scene by first establishing the viewing-coordinate **system**, also called the view reference coordinate **system**.



- **A** view plane, or projection plane, is then set up perpendicular to the viewing $\mathbf{z}_v$ axis. We can think of the view plane as the film plane in a camera that has been positioned and oriented for a particular shot of the scene.
- World-coordinate positions in the scene are transformed to viewing coordinates, then viewing coordinates are projected onto the view plane
- To establish the viewing-coordinate reference frame, we first pick a world coordinate position called the view reference point.

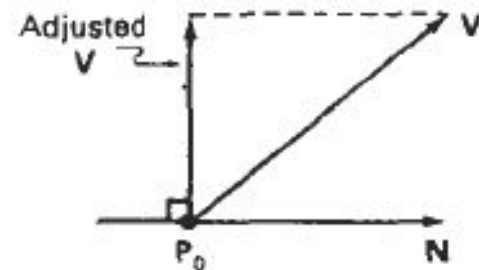
- This point is the origin of our viewing-coordinate system.
- The view reference point **is** often chosen to be close to or on the surface of some object in a scene.
- But we could also choose a point that is at the center of an object, or at the center of a group of objects, or somewhere out in front of the scene to be displayed.

- Next is **normal vector(N)**, it is perpendicular to view plane. we select the positive direction for the viewing z_v axis, and the orientation of the view plane, by specifying the view-plane normal vector, N.
- We choose a world-coordinate position, and this point establishes the direction for N relative either to the world origin or to the viewing-coordinate origin.

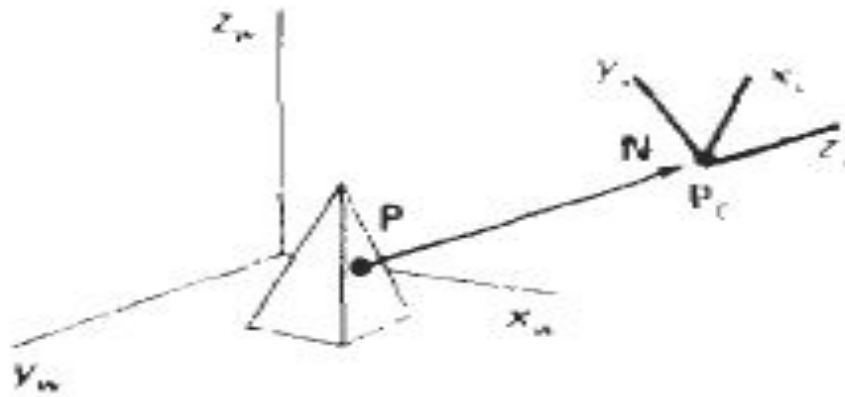


View-Up Vector(V):

- Once a view-plane normal vector N is chosen, the view-up vector V is used to establish the positive direction for the y-view axis.
- This vector helps define the orientation of the viewing plane and ensures that the scene is displayed correctly.



- ***Viewing distance(R):***
- The distance between the virtual camera and the object view reference point and view plain.



uvn Viewing-Coordinate Reference Frame

1. **n (View-Plane Normal Vector):**

- Points directly away from the viewer's eye.
- Perpendicular to the viewing plane.
- Defined by the coordinates (n_x, n_y, n_z) .

2. **u (Horizontal Vector):**

- Points to the right of the viewer.
- Perpendicular to both **n** and **v**.
- Defined by the coordinates (u_x, u_y, u_z) .

3. **v (Vertical Vector):**

- Points upwards in the viewer's field of view.
- Perpendicular to both **n** and **u**.
- Defined by the coordinates (v_x, v_y, v_z) .



$$\begin{aligned} \mathbf{n} &= \frac{\mathbf{N}}{|\mathbf{N}|} = (n_x, n_y, n_z) \\ \mathbf{u} &= \frac{\mathbf{V} \times \mathbf{n}}{|\mathbf{V} \times \mathbf{n}|} = (u_x, u_y, u_z) \\ \mathbf{v} &= \mathbf{n} \times \mathbf{u} = (v_x, v_y, v_z) \end{aligned}$$

Transformation from World to Viewing Coordinates

1. Translate the view reference point to the origin of the world-coordinate system.
2. Apply rotations to align the x_v , y_v , and z_v axes with the world x_w , y_w and z_w axes respectively.

Translate

- If the view reference point is specified at world position (x_0, y_0, z_0) , this point is translated to the world origin with the matrix transformation

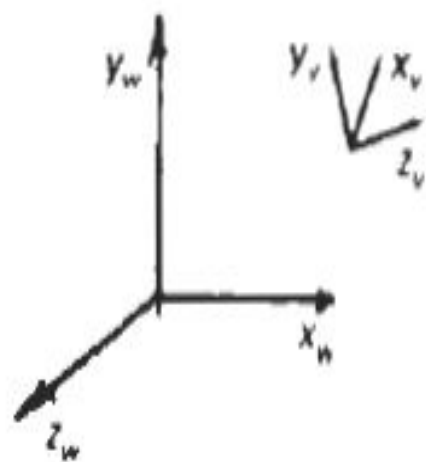


The diagram shows two 3D coordinate systems. On the left, a world coordinate system with axes x_w, y_w, z_w and a view coordinate system with axes x_v, y_v, z_v . The view system is translated such that its origin is at the world position (x_0, y_0, z_0) . On the right, the same view system is shown translated so its origin is at the world origin $(0, 0, 0)$. The transformation matrix T is given by:

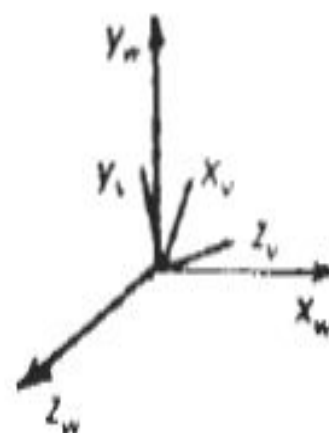
$$T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotations

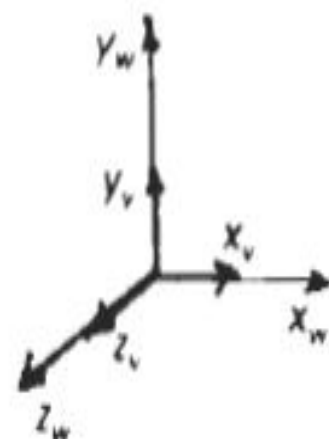
- The rotation sequence can require up to three coordinate-axis rotations, depending on the direction we choose for N .
- In general, if N is not aligned with an):
- work-coordinate axis, we can superimpose the viewing and world systems with the transformation sequence **RZ , RY , Rx** ,.
- That is, we first rotate around the world **xw** , axis to bring zv , into the **xw , zv** , plane.
- Then, we rotate around the world **yw** , axis to align the zv , and zw axes.
- The final rotation is about the **zw** , axis to align the **yw** , and yv . axes



(a)



(b)



(c)

- Another method for generating the rotation-transformation matrix is to calculate unit uvn vectors and form the composite rotation matrix directly.
- Given vectors $\mathbf{N}$ and $\mathbf{V}$, these unit vectors are calculated

$$\mathbf{n} = \frac{\mathbf{N}}{|\mathbf{N}|} = (n_1, n_2, n_3)$$

$$\mathbf{u} = \frac{\mathbf{V} \times \mathbf{N}}{|\mathbf{V} \times \mathbf{N}|} = (u_1, u_2, u_3)$$

$$\mathbf{v} = \mathbf{n} \times \mathbf{u} = (v_1, v_2, v_3)$$

- This method also automatically adjusts the direction for V so that v is perpendicular to n.
- The composite rotation matrix for the viewing transformation is then

$$R = \begin{bmatrix} u_1 & u_2 & u_3 & 0 \\ v_1 & v_2 & v_3 & 0 \\ n_1 & n_2 & n_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R = \begin{bmatrix} u_1 & u_2 & u_3 & 0 \\ v_1 & v_2 & v_3 & 0 \\ n_1 & n_2 & n_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{WC, VC} = R \cdot T$$

$$= \begin{bmatrix} u_x & u_y & u_z & -u \cdot P_0 \\ v_x & v_y & v_z & -v \cdot P_0 \\ n_x & n_y & n_z & -n \cdot P_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$-u \cdot P_0 = -x_0 u_x - y_0 u_y - z_0 u_z$$

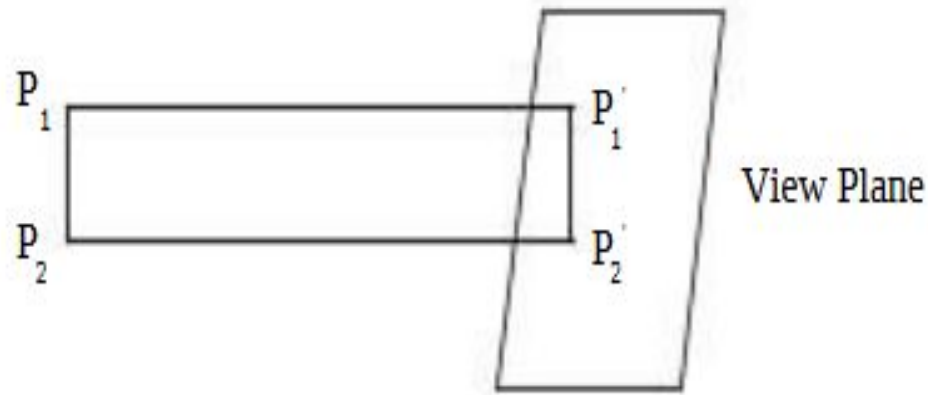
$$-v \cdot P_0 = -x_0 v_x - y_0 v_y - z_0 v_z$$

$$-n \cdot P_0 = -x_0 n_x - y_0 n_y - z_0 n_z$$

PROJECTIONS

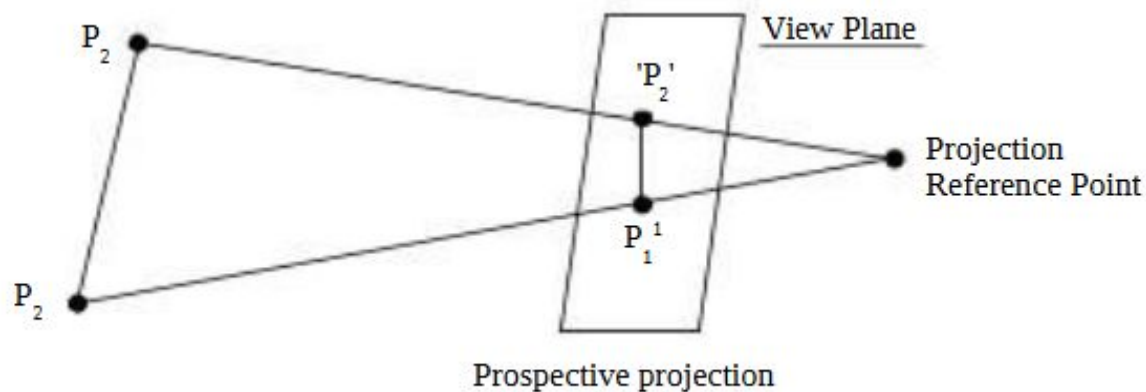
- Once world co-ordinate description of the objects in a scene are converted to viewing co-ordinates, we can project the three dimensional objects onto the two dimensional view plane. There are two basic projection methods:
 - Parallel projection
 - Prospective projection

- **Parallel projection:** In parallel projection, co-ordinates positions are transformed to the view plane along parallel lines.



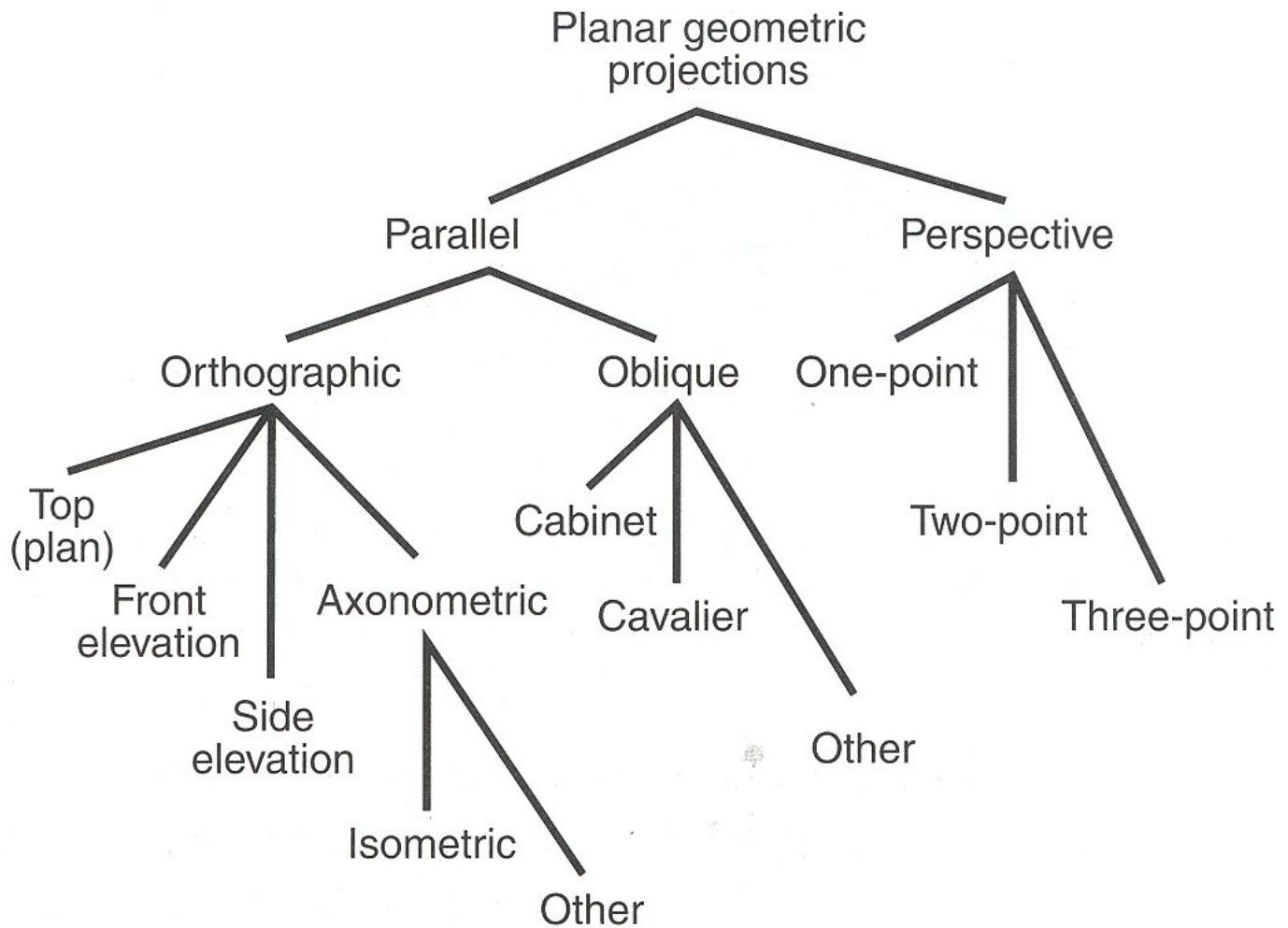
- Computer Graphics/ 3D A parallel projection preserve relative proportions of objects and this is the method used in drafting to produce scale drawing of three-dimensional objects.
- Accurate view of various sides of 3D object is obtained with parallel projection. But it does not given a realistic appearance of a 3D-object.

- **Prospective projection:** In prospective projection, objects positions are transformed to the view plane along lines that converge to a point called projection reference point (center of projection).
- The projected view of an object is determined by calculating the intersection of the projection lines with the view plane.

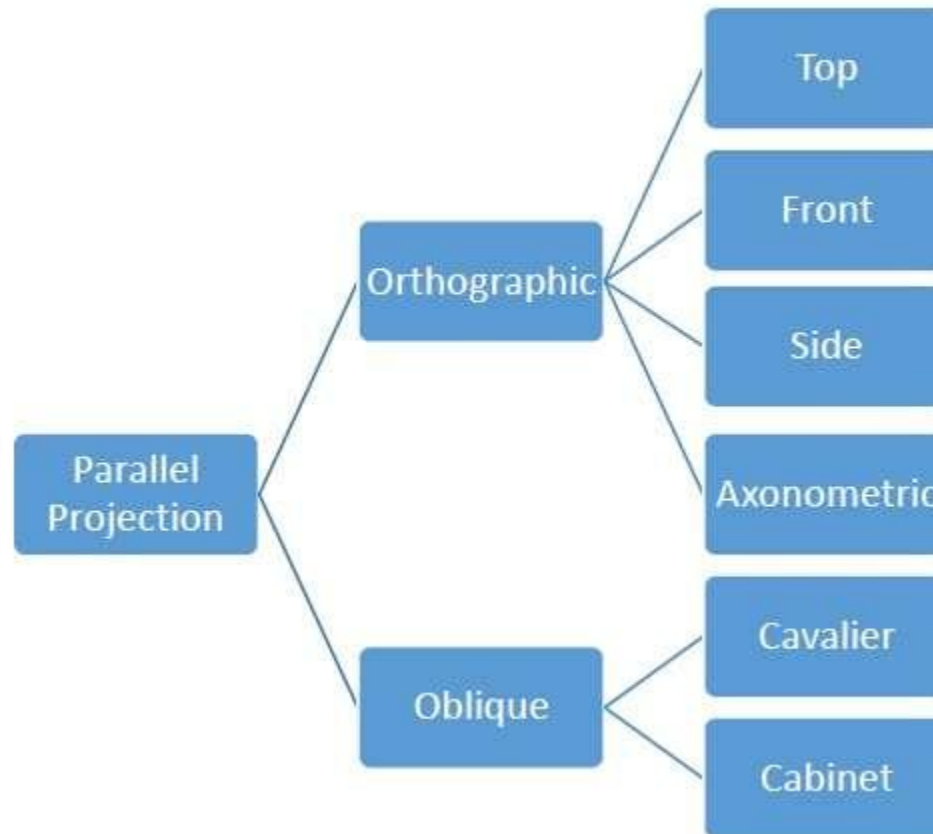


- A prospective projection, on the other hand, produces realistic views but does not preserve relative proportions.
- Projections of distance objects from view plane are smaller than the projections of objects of the same size that are closer to the projection place.

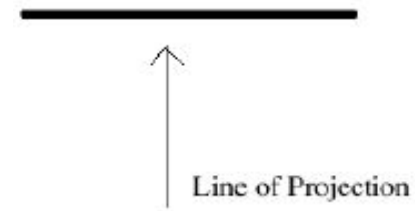
Taxonomy of Planar Geometric Projections

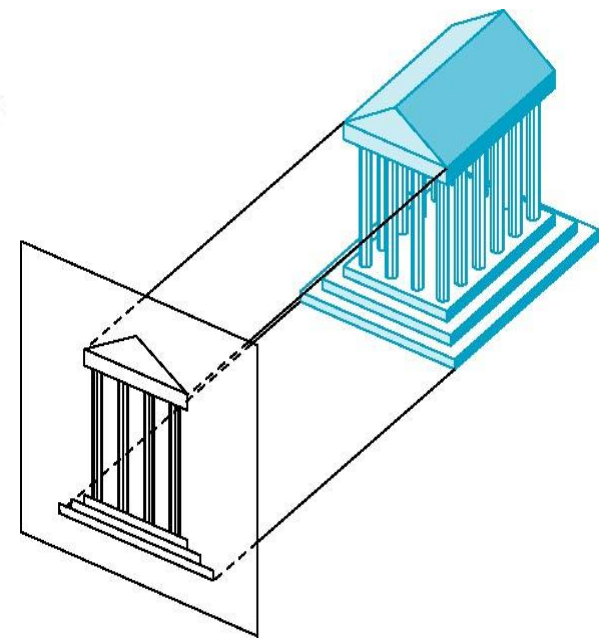
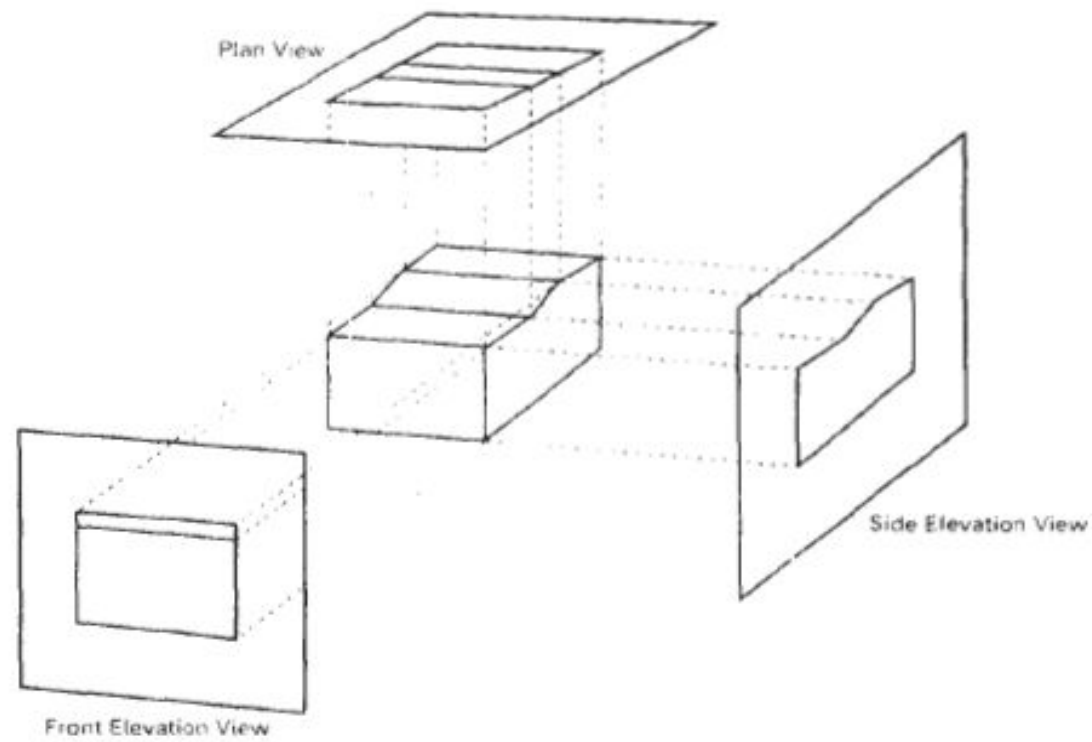


Parallel projection



- When the projection is perpendicular to the view plane, we have an orthographic parallel projection. Otherwise, we have an oblique parallel projection.
- Orthographic projections are used to obtain the front, side and top views of an object.





- Arguably the **simplest projection**
 - Image **plane is perpendicular** to one of the coordinate axes;
 - **Project onto plane** by dropping that coordinate;
 - All **rays are parallel**.

Parallel Projection - Advantages and Disadvantages

Are actually a kind of **affine transformation**

- Preserves **distances**
- **Shapes preserved**
- Parallel lines remain **parallel**
- **Ratios** are preserved
- Angles not (in general) preserved
- **Not realistic looking**
 - because many **surfaces hidden** from view
 - Often we add the isometric
- **Good for exact measurements**
 - Building plans ,Manuals, CAD, architectural drawings

Find the orthographic projection of a unit cube onto the $x=0$, $y=0$ and $z=0$ plane.

X=0 plane projection /YZ plane projection

$A(0,0,0) \square A'(0,0,0)$

$B(0,0,1) \square B'(0,0,1)$

$C(1,0,1) \square C'(0,0,1)$

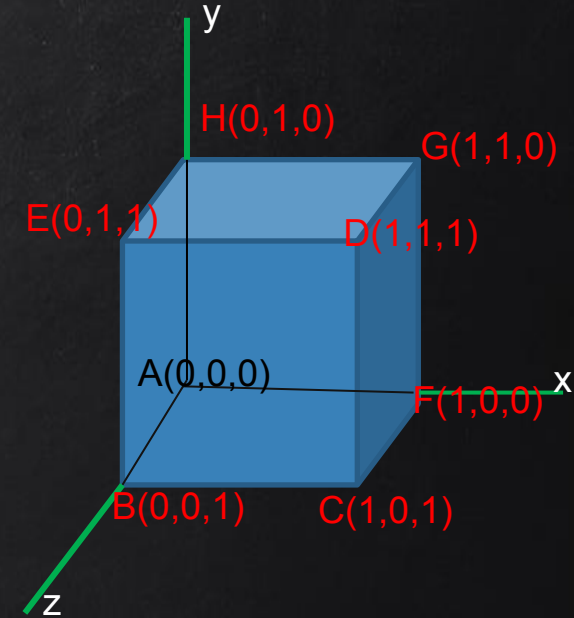
$D(1,1,1) \square D'(0,1,1)$

$E(0,1,1) \square E'(0,1,1)$

$F(1,0,0) \square F'(0,0,0)$

$G(1,1,0) \square G'(0,1,0)$

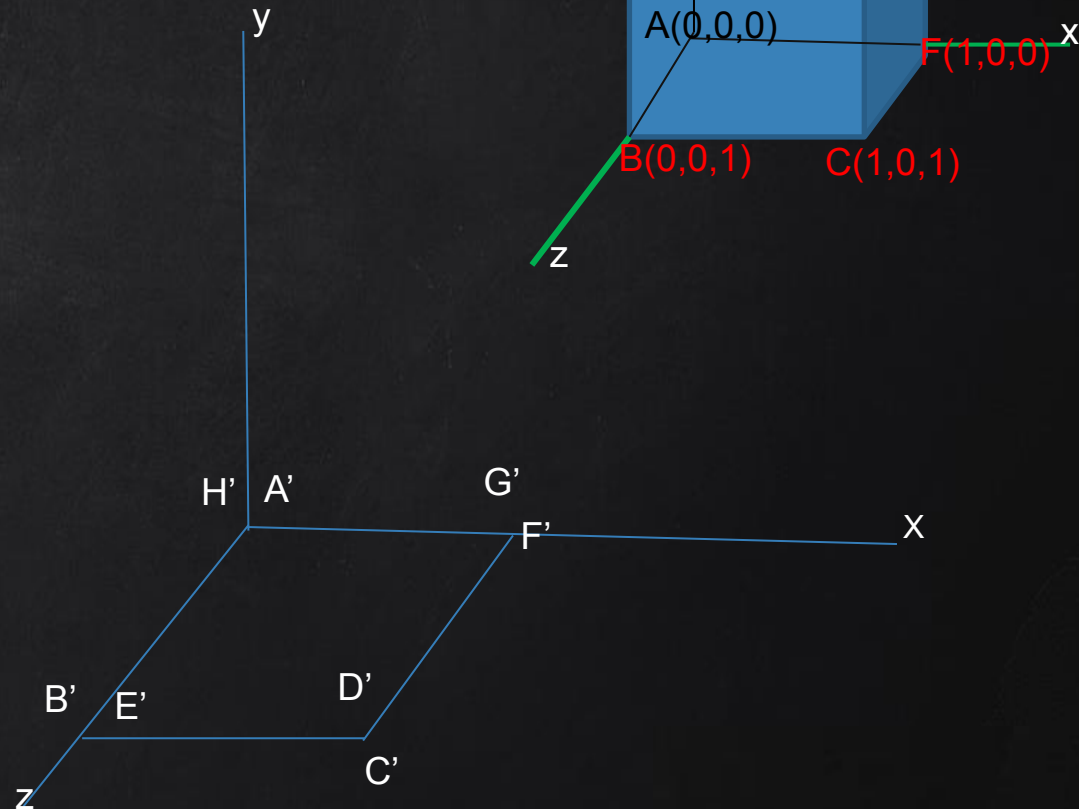
$H(0,1,0) \square H'(0,1,0)$



Find the orthographic projection of a unit cube onto the $x=0$, $y=0$ and $z=0$ plane.

Y=0 plane projection /XZ plane projection

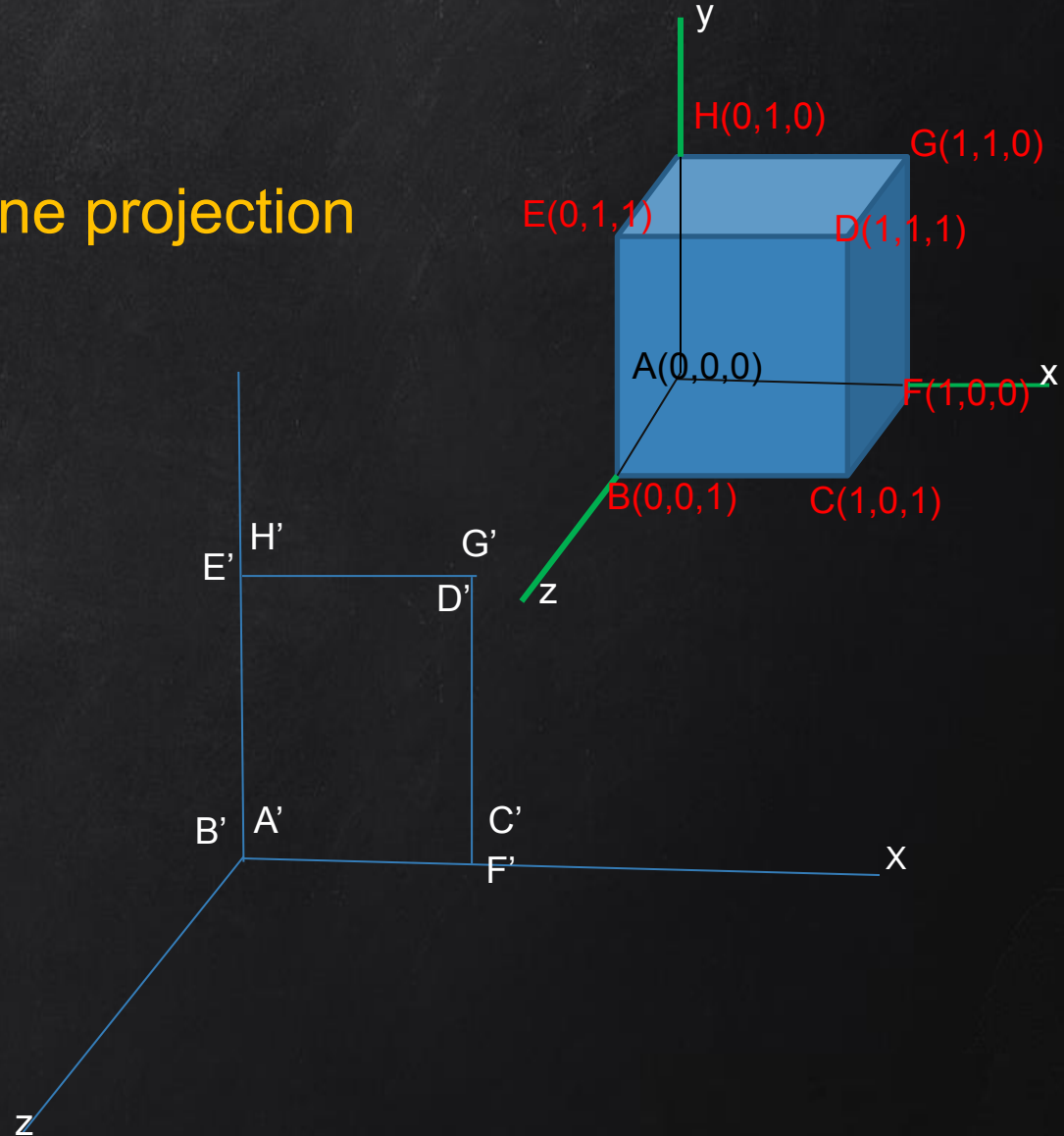
$A(0,0,0) \square A'(0,0,0)$
 $B(0,0,1) \square B'(0,0,1)$
 $C(1,0,1) \square C'(1,0,1)$
 $D(1,1,1) \square D'(1,0,1)$
 $E(0,1,1) \square E'(0,0,1)$
 $F(1,0,0) \square F'(1,0,0)$
 $G(1,1,0) \square G'(1,0,0)$
 $H(0,1,0) \square H'(0,0,0)$



Find the orthographic projection of a unit cube onto the $x=0$, $y=0$ and $z=0$ plane.

Z=0 plane projection /XY plane projection

$A(0,0,0) \rightarrow A'(0,0,0)$
 $B(0,0,1) \rightarrow B'(0,0,0)$
 $C(1,0,1) \rightarrow C'(1,0,0)$
 $D(1,1,1) \rightarrow D'(1,1,0)$
 $E(0,1,1) \rightarrow E'(0,1,0)$
 $F(1,0,0) \rightarrow F'(1,0,0)$
 $G(1,1,0) \rightarrow G'(1,1,0)$
 $H(0,1,0) \rightarrow H'(0,1,0)$

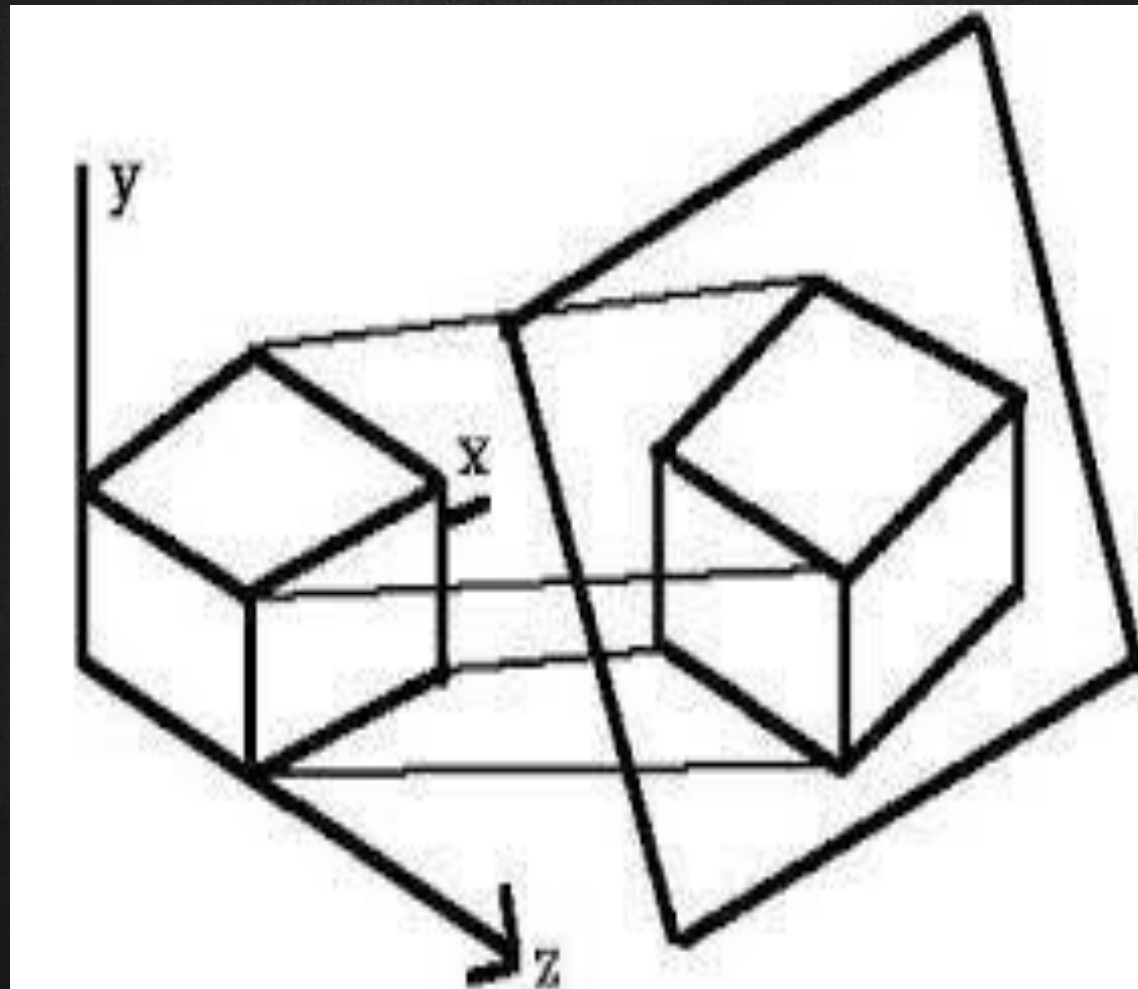


Isometric Projections

Orthographic projections that show more than one side of an object are called **axonometric orthographic projections**.

The most common axonometric projection is an **isometric projection** where the projection plane intersects each coordinate axis in the model coordinate system at an equal distance.

In this projection parallelism of lines are preserved but angles are not preserved.



Axonometric - Advantages and Disadvantages

- Lines are *scaled (foreshortened)* but can find scaling factors
- Lines preserved, but angles not preserved
- Projection of a circle in a plane not parallel to the projection plane is an ellipse
- Can see three principal faces of a box-like object
- Some optical illusions possible
- Parallel lines appear to diverge
- Does not look real because far objects are scaled the same as near objects
- Used in CAD applications

Oblique Projection

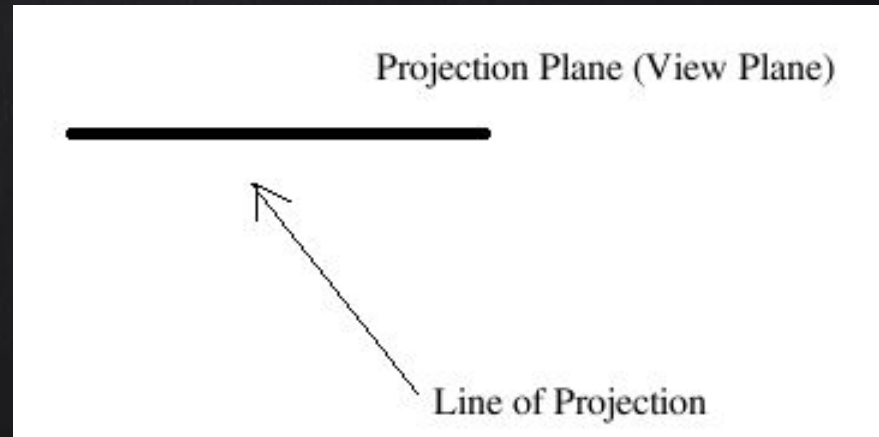
Projecting the plane along parallel lines that are not perpendicular to the projection plane.

There are two types of oblique projection, these are: **Cavalier Projection**

Cabinet Projection

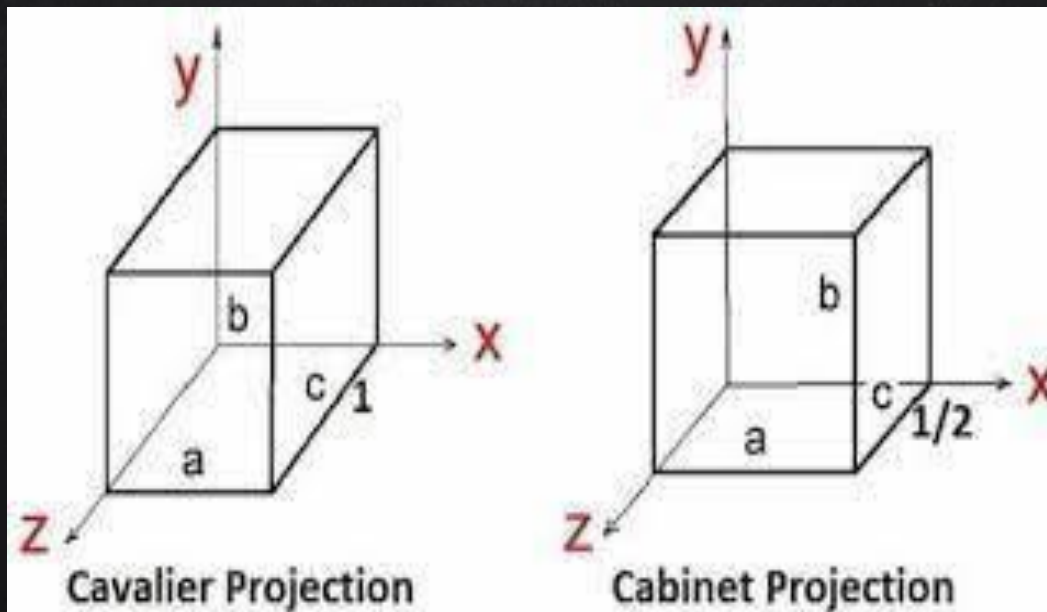
The cavalier projection preserves the lengths of lines that are perpendicular to the projection plane.

Projection line makes an angle 45 degree with projection plane .

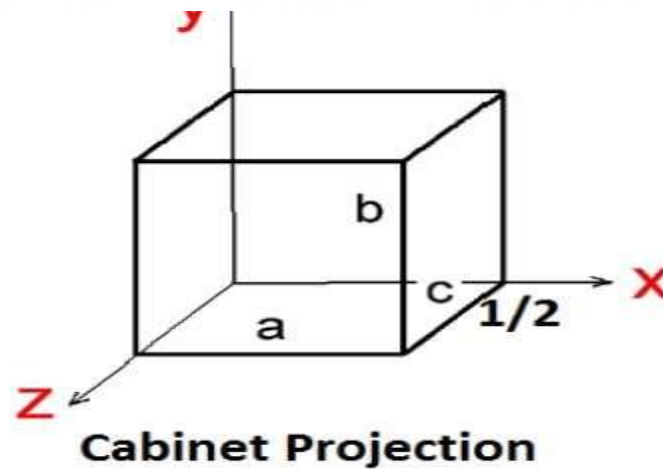
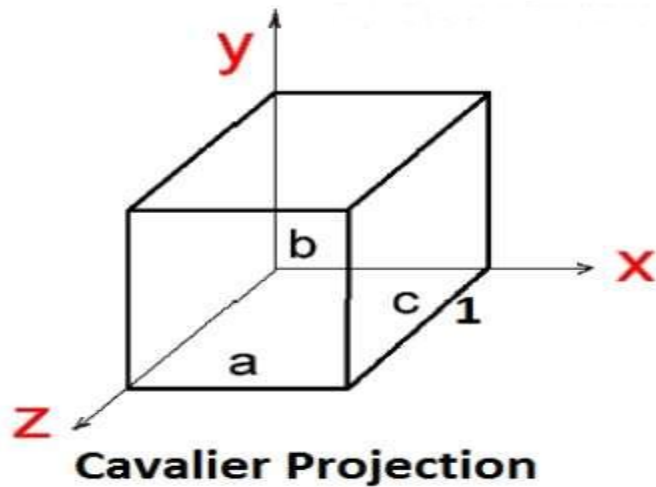
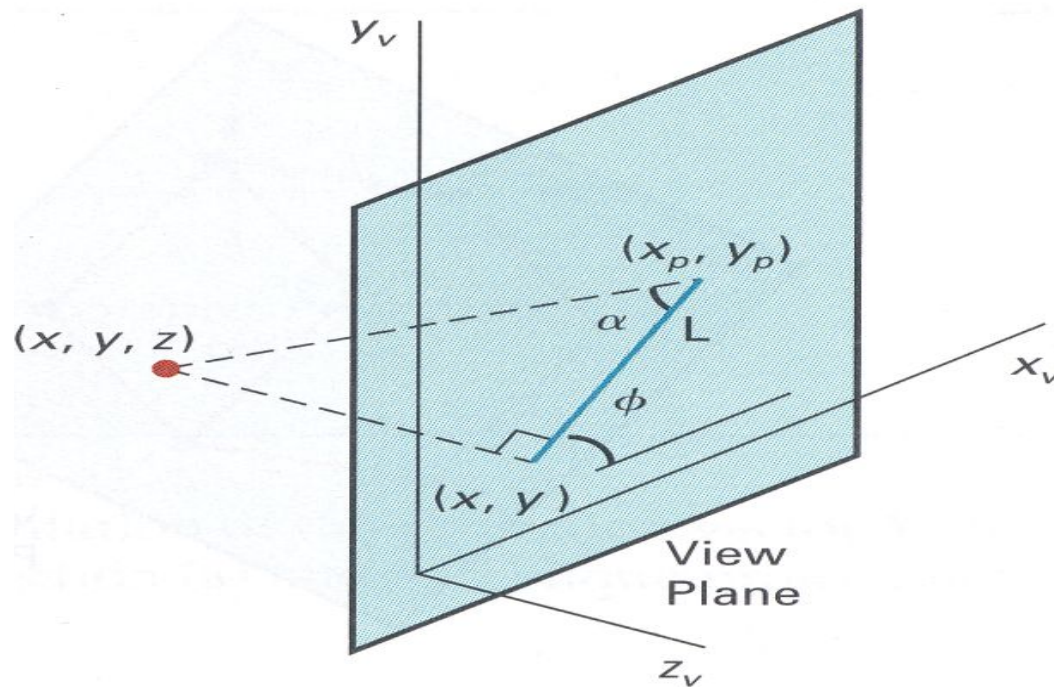


Cabinet projection: Line perpendicular to the viewing surface are projected at one –half of Original size. Projection line makes an angle 63.4 degree with projection plane .

Cabinet projections appear more realistic than cavalier projections because of reduction in the length of perpendiculars .



Oblique projection



To produce an oblique parallel projection of a 3D object, we need to specify the angles α and ϕ as shown above.

In the figure above, oblique parallel projection of a point (x, y, z) on a 3D object is shown. The point is projected on to (x_p, y_p) on the view plane.

The oblique parallel projection is the line joining point (x, y, z) and the point (x_p, y_p) .

From the figure above,

$$\cos \phi = \frac{x_p - x}{L}$$

$$x_p = x + L \cos \phi \quad \Rightarrow 1$$

$$\sin \phi = \frac{y_p - y}{L}$$

$$y_p = y + L \sin \phi \quad \Rightarrow 2$$

$$\tan \alpha = \frac{z - 0}{L}$$

$$L = \frac{z}{\tan \alpha}$$

Let $L1 = \frac{1}{\tan \alpha}$

Then,

$$L = z.L1$$

Then Equation 1 and 2 become

$$x_p = x + z.L1.Cos \phi$$

$$y_p = y + z.L1.Sin \phi$$

In matrix form,

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & L1.Cos\phi & 0 \\ 0 & 1 & L1.Sin\phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where $L1 = \frac{1}{\tan \alpha}$

The matrix $M_{Parallel}$ is called Parallel Projection matrix.

The complete transformation matrix for projecting a 3D object defined in world coordinate system to a 2D screen is

$$M_{Parallel}.R.T$$

Thus the equation for parallel projection is

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M_{Parallel}.R.T. \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The matrices R and T are those discussed in the section 'Viewing coordinate system'.

Special Cases:

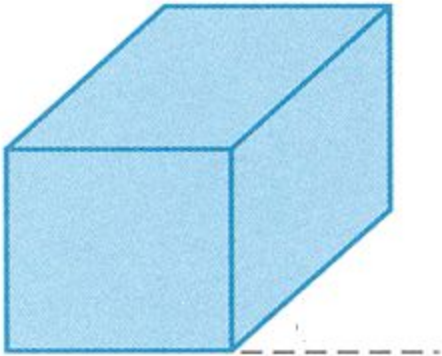
When the value of α is 90^0 , we get Orthographic Parallel projection. ($\tan 90 = \text{infinity}$, $\frac{1}{\tan 90} = 0$, means $L1 = 0$.)

When the value of α is 45^0 , ($\tan 45 = 1$), we get Cavalier Parallel projection.

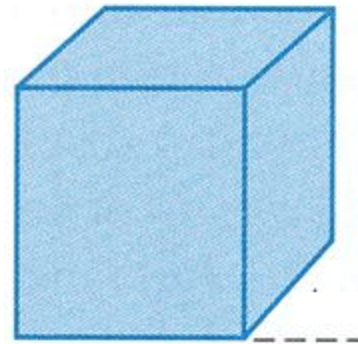
Oblique Projections

- **Cavalier Projection** - An oblique projection obtained when the angle between oblique projectors and the plane of projection, $\alpha = 45^\circ$.
- In a cavalier projection the foreshortening factors for all three principal direction are equal. The resulting figure appears too thick. A cabinet projection is used to correct this deficiency.
- **Cabinet Projection** - An oblique projection for which the foreshortening factor for edges perpendicular to the plane of projection is one half.
- $\alpha = 63.4^\circ$ (angle between oblique projectors and the plane of projection)

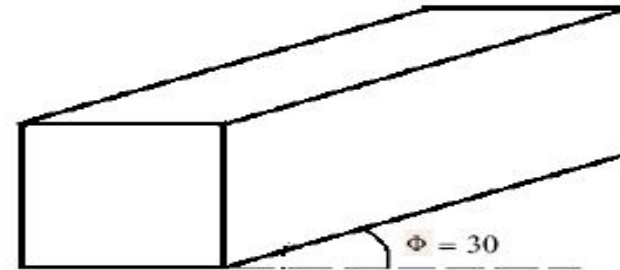
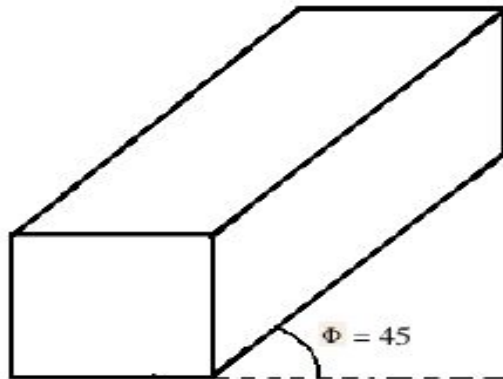
•C more realistic



Cavalier
(DOP $\alpha = 45^\circ$)
 $\tan(\alpha) = 1$
Depth = W & H



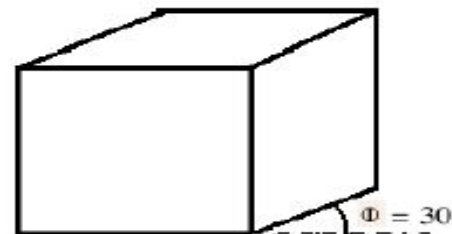
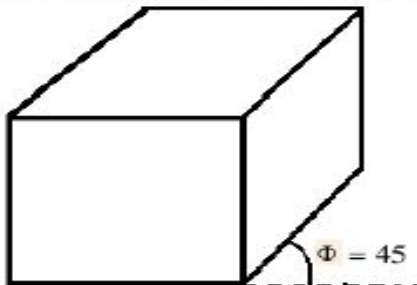
Cabinet
(DOP $\alpha = 63.4^\circ$)
 $\tan(\alpha) = 2$
Depth = $\frac{1}{2}$ (W & H)



Cavalier Projection ($\alpha = 45^\circ$)

Parallel Projection of a box

When the value of $\tan \alpha$ is 2, we get Cabinet Parallel projection.

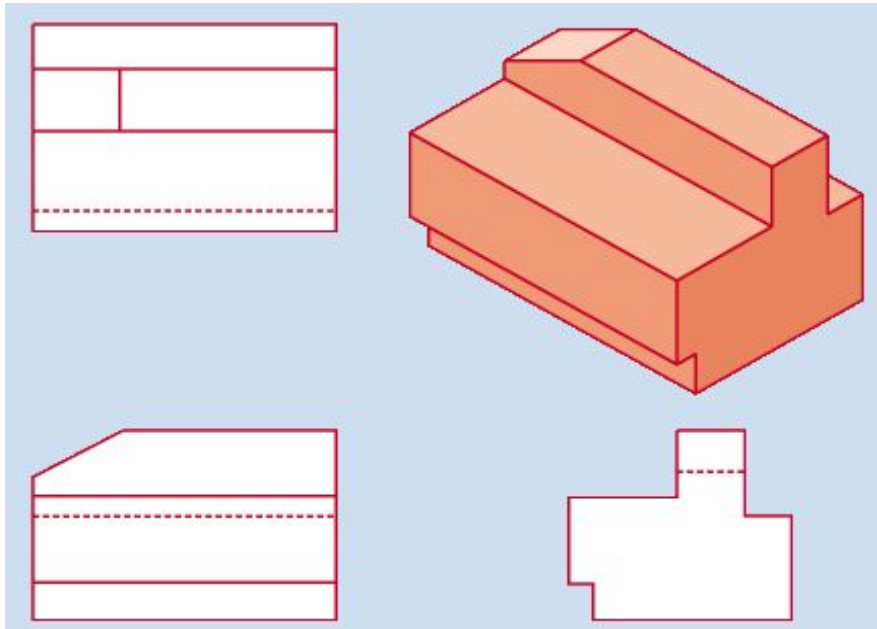


Cabinet Projection ($\tan \alpha = 2$)

Parallel Projection of a cube

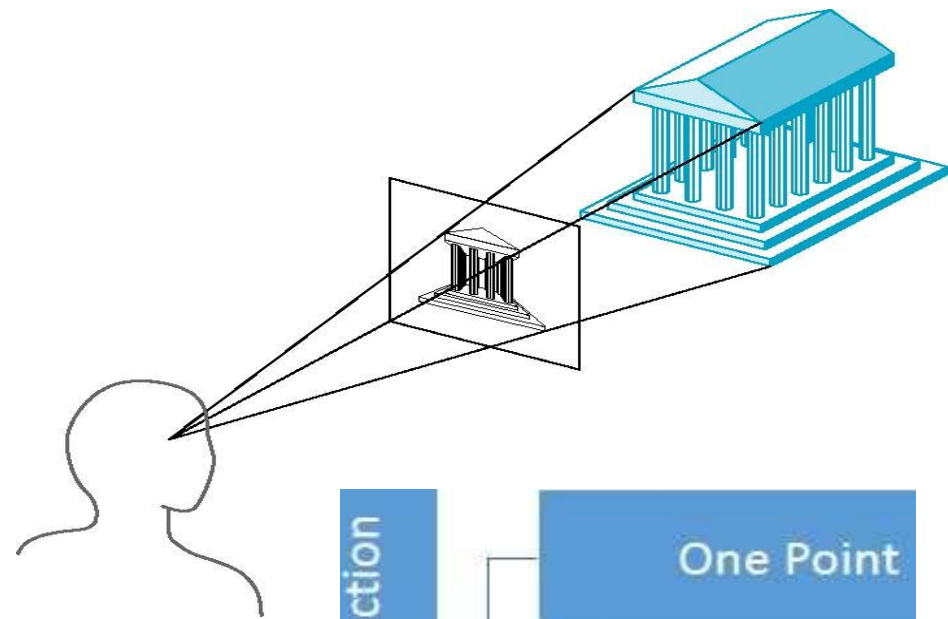
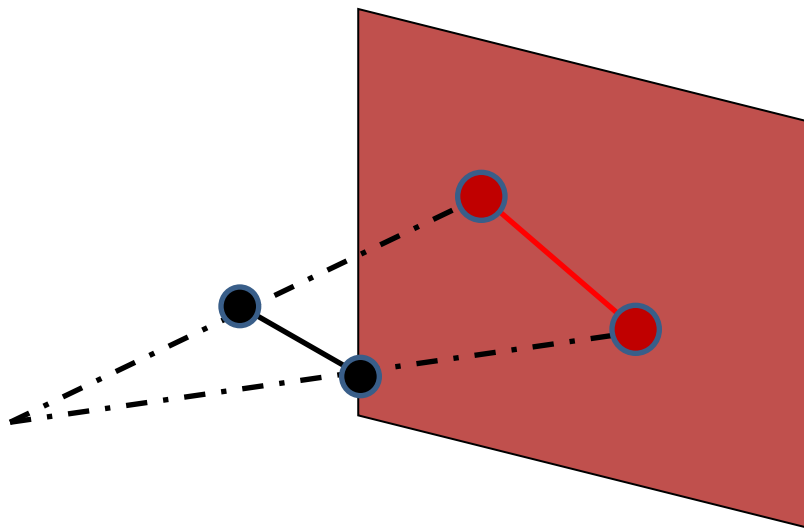
Advantages and Disadvantages

- Can pick the angles to emphasize **a particular face**
 - Architecture: plan oblique, elevation oblique
- Angles in faces parallel to projection plane are preserved while we can still see “around” side
- In physical world, cannot create with simple camera; possible with bellows camera or special lens (architectural)

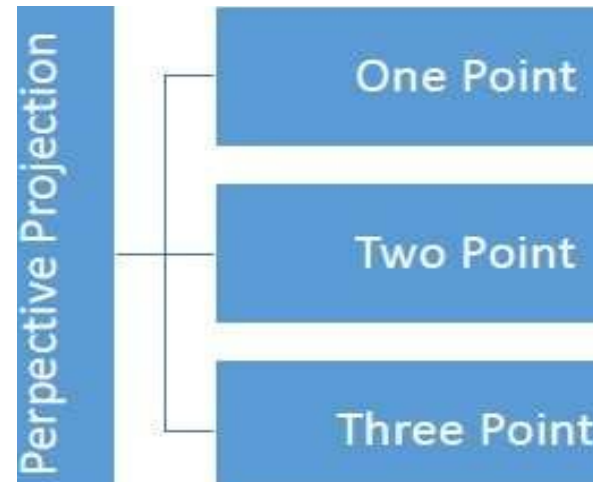


Perspective Projection

Projectors converge at **projection reference point or center of projection (COP)**

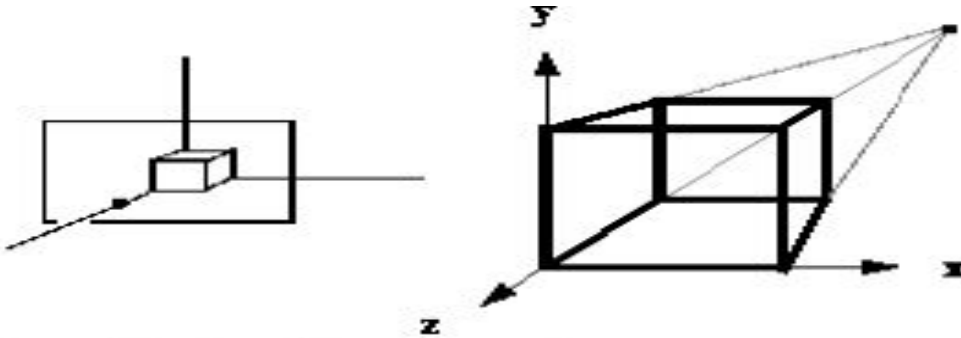


- **Naturally we see things in perspective**
- **Objects appear smaller**, the farther away they are;
- Rays from view point **are not parallel**.

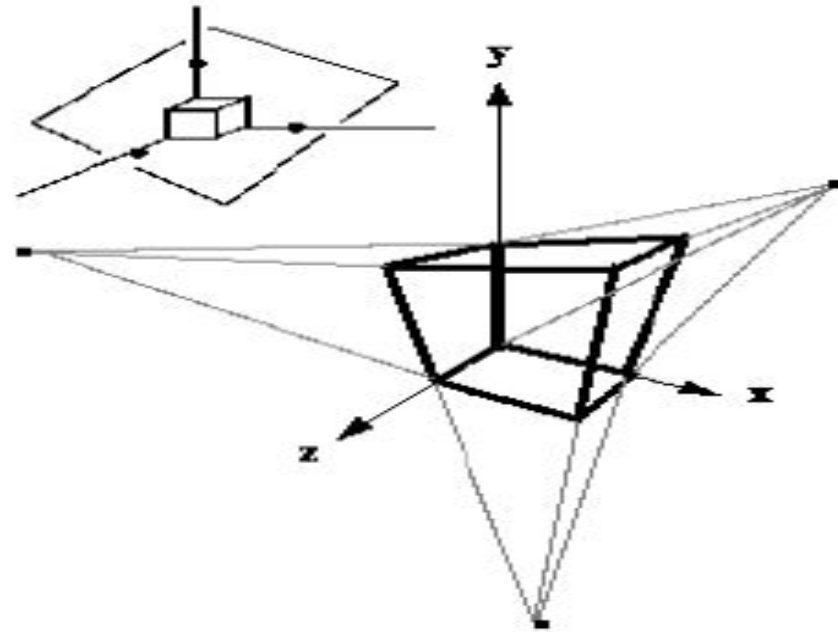


Vanishing Points

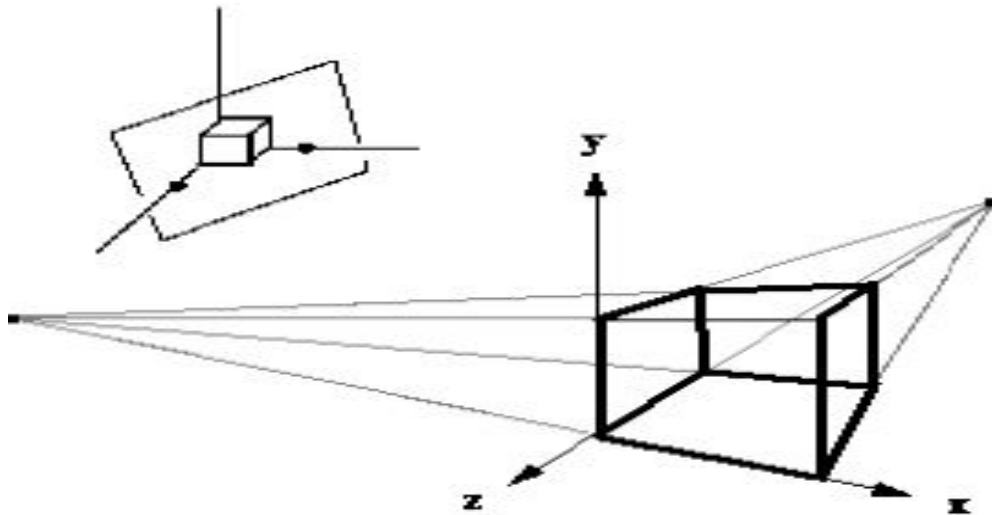
- Parallel lines (not parallel to the projection plane) on the object converge at a single point in the projection (*vanishing point*)
- Drawing simple perspectives by hand uses these vanishing point(s)



One Point Perspective
(z-axis vanishing point)



Three Point Perspective
(z, x, and y-axis
vanishing points)

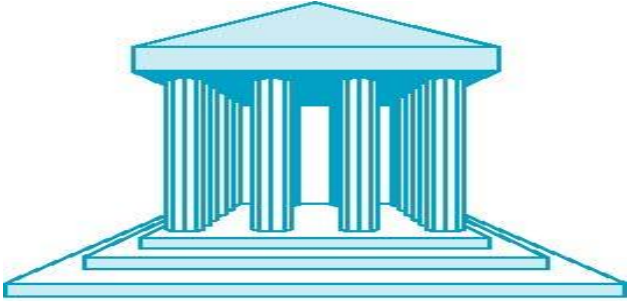


Two Point Perspective
z, and x-axis vanishing points

Types of Perspectives

❖ One-Point Perspective

- One principal face parallel to projection plane
- One vanishing point for cube



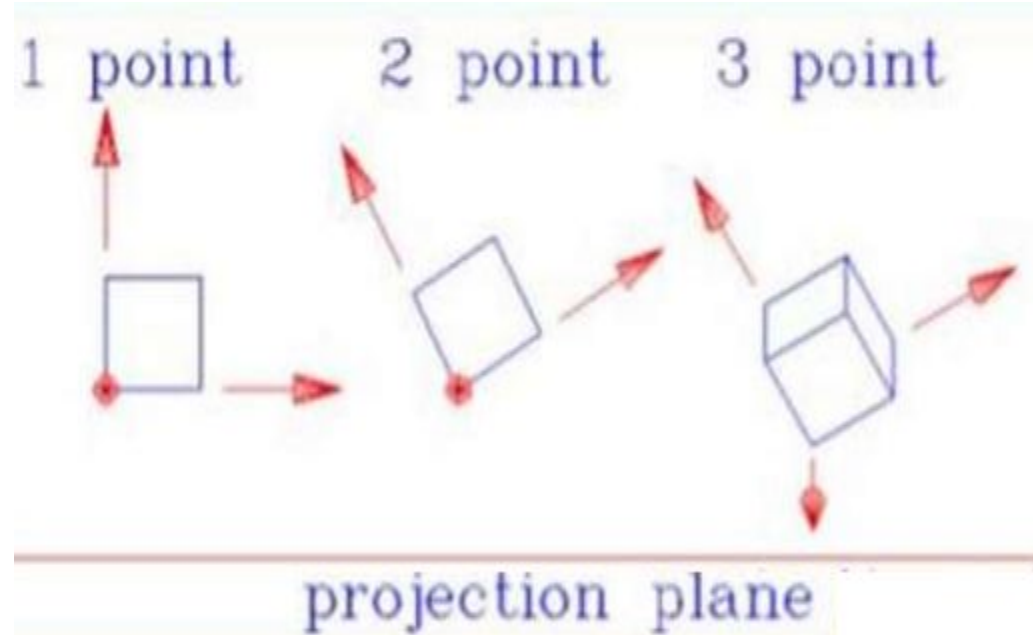
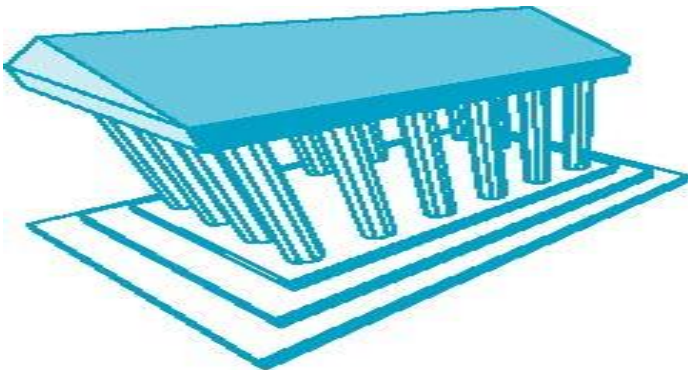
❖ Two-Point Perspective

- One principal direction parallel to projection plane
- Two vanishing points for cube

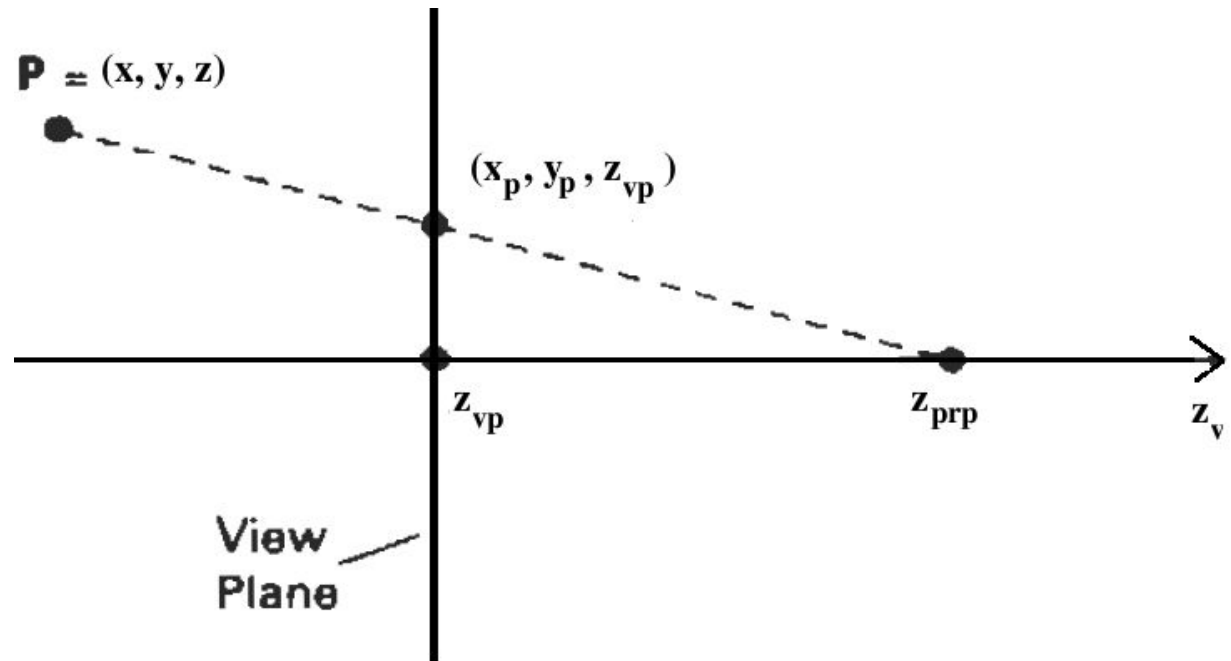


❖ Three-Point Perspective

- No principal face parallel to projection plane
- Three vanishing points for cube



- Let us derive the equation for perspective projection.
- In this derivation, we consider the projection reference point to be along the z_v axis.



- Let the projection reference point is at position Z_{prp} along Z_v axis and view plane is at Z_{vp} :
- Let $P(x, y, z)$ is a point on the 3D object and $(x_p; y_p; z_p)$ is the projection of this point on the view plane.
- The equation for the projection line can be written in parametric form as,

$$x' = x - xu$$

$$y' = y - yu$$

$$z' = z - (z - z_{pRP})u$$

$$z' = z_{vp} \Rightarrow u = \frac{z_{vp} - z}{z_{pRP} - z}$$

$$x_p = x \cdot \left[\frac{z_{pRP} - z_{vp}}{z_{pRP} - z} \right] = x \cdot \left[\frac{d_p}{z_{pRP} - z} \right]$$

$$y_p = y \cdot \left[\frac{z_{pRP} - z_{vp}}{z_{pRP} - z} \right] = y \cdot \left[\frac{d_p}{z_{pRP} - z} \right]$$

$$d_p = z_{pRP} - z_{vp}$$

Equations 4, 5 and 6 can be written in matrix form as,

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-z_{vp}}{d_p} & z_{vp}(\frac{z_{prp}}{d_p}) \\ 0 & 0 & \frac{-1}{d_p} & \frac{z_{prp}}{d_p} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where $h = \frac{z_{prp} - z}{d_p}$

$$x_h = x_p \cdot h$$

$$y_h = y_p \cdot h$$

$$d_p = z_{prp} - z_{vp}$$

d_p is the distance of the view plane from projection reference point, z_{prp} .

The above is the equation for perspective projection for a point (x,y,z) on a 3D object for the special case that projection reference point is along the z_v axis.

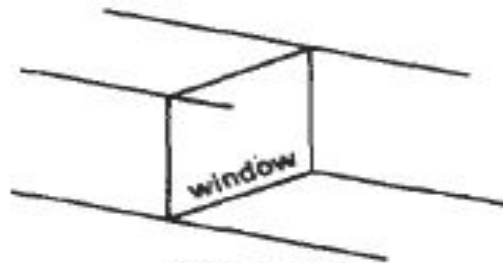
Perspective Projection - Advantages and Disadvantages

- Objects farther from viewer are projected smaller than the same sized objects closer to the viewer (*diminution*)
 - Looks realistic (size varies inversely with distance)
- Equal distances along a line are projected into unequal distances (*nonuniform foreshortening*)
- Angles preserved only in planes parallel to the projection plane
- More difficult to construct by hand than parallel projections (but not more difficult by computer)
- Exact measurements cannot be taken
 - Lines map to lines
 - Parallel lines do not necessarily remain parallel
 - Ratios are not preserved

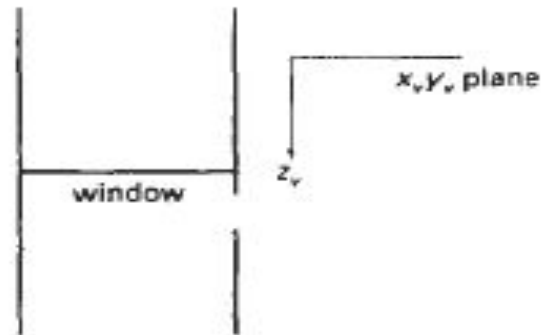
View volume

- When we look on to a 3D scene, only some parts or some objects in the 3D scene are visible to our eyes.
- The objects or parts behind us are not visible. Some parts on the our left, right, bottom and top sides are not visible.
- The volume of space that is actually rendered into the image is called the **view volume**. Things inside the view volume make it into the image; things that are not in the view volume are clipped and cannot be seen.

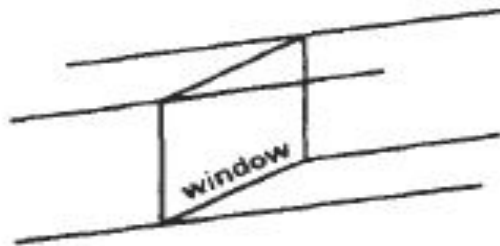
- The size of the view volume depends on the size of the window, while the shape of the view volume depends on the type of projection to be used to generate the display.
- In any case, four sides of the volume are planes that pass through the edges of the window. For a parallel projection, these four sides of the view volume form an infinite parallelepiped



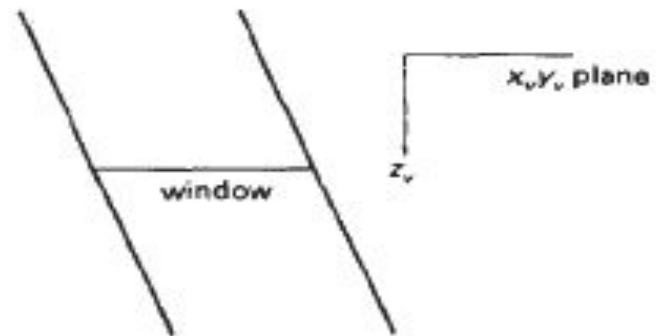
Orthographic-
Projection
View Volume
(a)



Orthographic-
Projection
View Volume
(b)

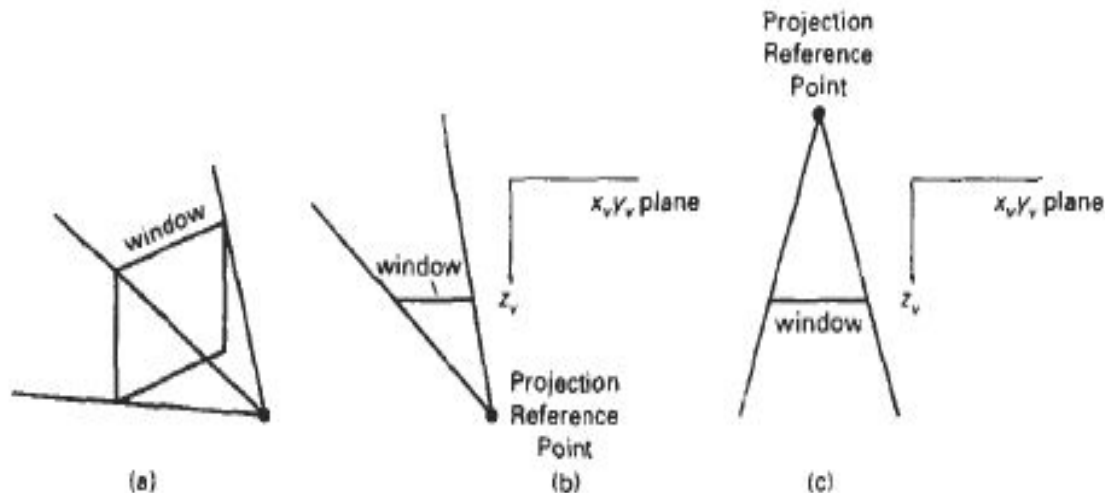


Oblique-
Projection
View Volume
(c)

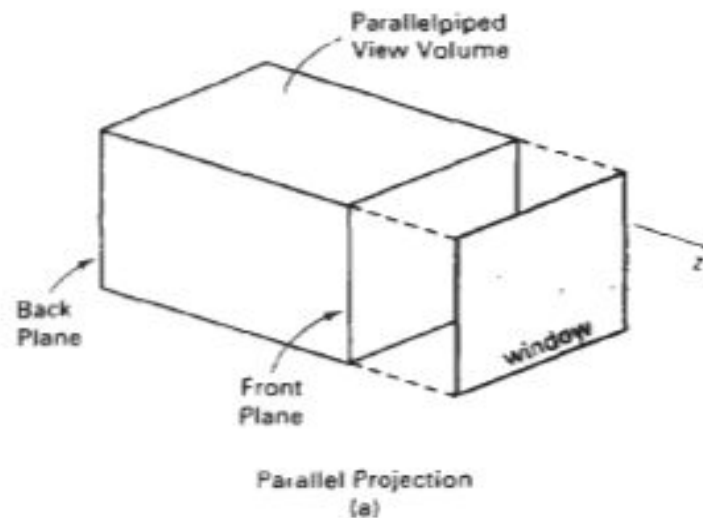


Oblique-
Projection
View Volume
(d)

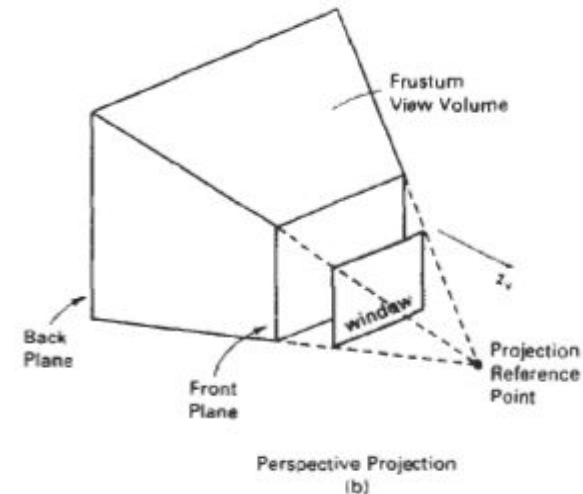
- For a perspective projection, the view volume is-a pyramid with apex at the projection reference point



- View volume with respect to parallel projection is termed as **parallel-piped**.



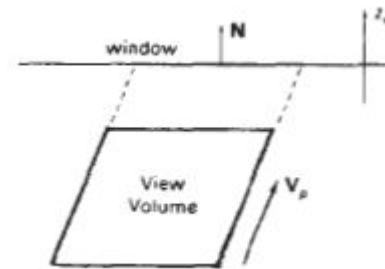
- View volume with respect to perspective projection is termed as **frustum**.



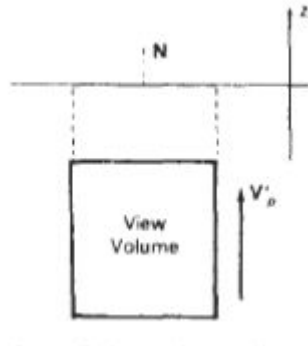
- **Near Plane:** The near plane is the closest clipping plane of the frustum.
- **Far Plane:** The far plane is the farthest clipping plane of the frustum.

General Parallel-Projection Transformations

- The direction of a parallel projection is specified with a projection vector from the projection reference point to the center of the view window.
- The general shape of a finite view volume for a given projection vector and projection window in the view plane.



- The oblique projection transformation with a shear operation that converts the view volume in to the regular parallelepiped



- If the projection vector is specified in world coordinates, it must first be transformed to viewing coordinates using the rotation **matrix**

- We need to determine the elements of a shear matrix that will align the projection vector **Vp** with the view plane normal vector **N**
- This transformation can be expressed as

$$\mathbf{V}'_p = \mathbf{M}_{\text{parallel}} \cdot \mathbf{V}_p$$

$$= \begin{bmatrix} 0 \\ 0 \\ p_z \\ 0 \end{bmatrix}$$

- Where **M**_{parallel} equivalent to the parallel projecton matrix and represents a z-axis shear of the form

$$M_{\text{parallel}} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The explicit transformation equations from in terms of shear parameters a and b are

$$0 = p_x + a p_z$$

$$0 = p_y + b p_z$$

- so that the values for the shear parameters are

$$a = -\frac{p_x}{p_z}, \quad b = -\frac{p_y}{p_z}$$

- Thus, we have the general parallel-projection matrix in terms of the elements of the projection vector as

$$\mathbf{M}_{\text{parallel}} = \begin{bmatrix} 1 & 0 & -p_x/p_z & 0 \\ 0 & 1 & -p_y/p_z & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

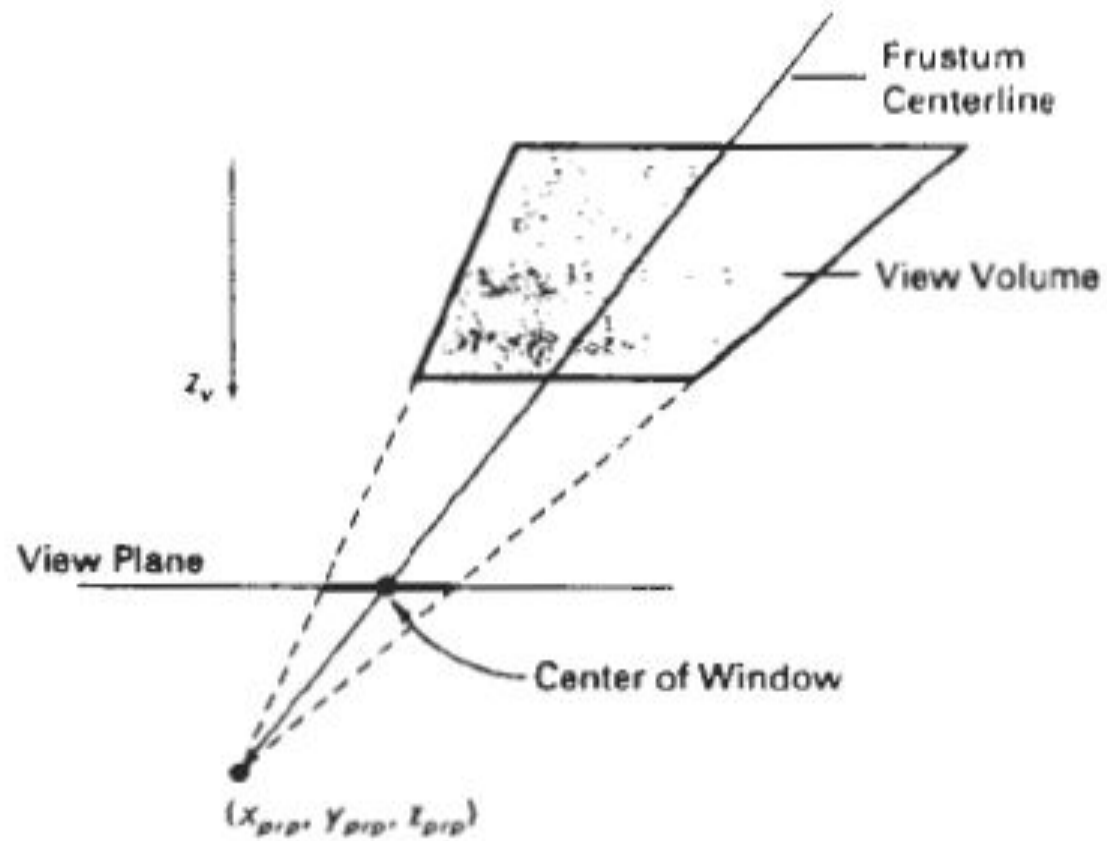
- **This** matrix is then concatenated with transformation **R.T** to produce the transformation from world coordinates to parallel-projection coordinates.
- For an orthographic parallel projection, **p_x** , = **$p_y= 0$** , and **Mparallel** is the identity matrix.
- We can relate the components of the projection vector to parameters **L**, **α** and **φ**

$$\frac{L \cos \phi}{z} = -\frac{p_x}{p_z}$$

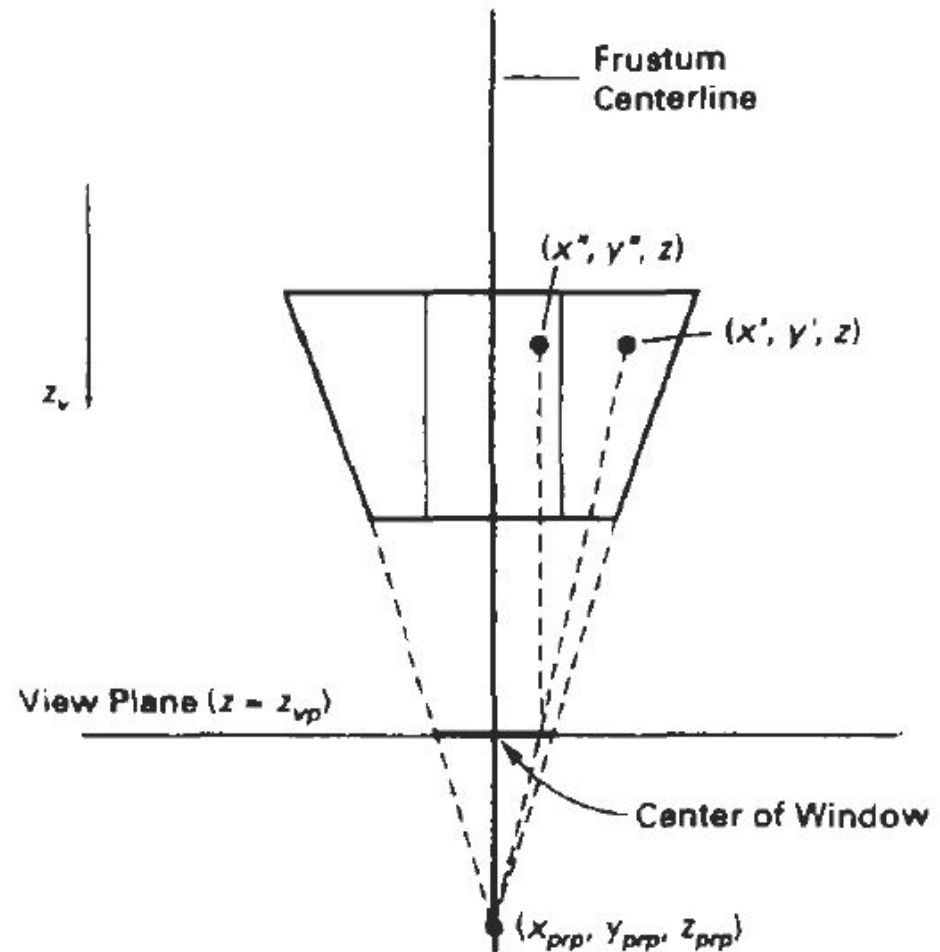
$$\frac{L \sin \phi}{z} = -\frac{p_y}{p_z}$$

General Perspective-Projection Transformations

- The projection reference point can be located at any position in the viewing system, except on the view plane or between the front and back clipping planes.
- We can obtain the general perspective-projection transformation with the following two operations:
 1. Shear the view volume **so** that the centerline of the frustum is perpendicular to the view plane.
 2. Scale the view volume with a scaling factor that depends on $1/z$.



- A shear operation to align a general perspective view volume with the projection window is shown



- This transformation has the effect of shifting all positions that lie along the frustum centerline, including the window center, to a line perpendicular to the view plane.
- With the projection reference point at a general position ***x_{prp} , y_{prp} , z_{prp}*** the transformation involves a combination ***Z*** -axis shear and a translation:

$$\mathbf{M}_{\text{shear}} = \begin{bmatrix} 1 & 0 & a & -a z_{prp} \\ 0 & 1 & b & -b z_{prp} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where the shear parameters are

$$a = - \frac{x_{prp} - (xw_{\min} + xw_{\max})/2}{z_{prp}}$$

$$b = - \frac{y_{prp} - (yw_{\min} + yw_{\max})/2}{z_{prp}}$$

- Points within the view volume are transformed by this operation as

$$x' = x + a(z - z_{prp})$$

$$y' = y + b(z - z_{prp})$$

$$z' = z$$

$$x'' = x' \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + x_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right)$$

$$y'' = y' \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + y_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right)$$

and the homogeneous matrix representation is

$$\mathbf{M}_{\text{scale}} = \begin{bmatrix} 1 & 0 & \frac{-x_{prp}}{z_{prp} - z_{vp}} & \frac{x_{prp}z_{vp}}{z_{prp} - z_{vp}} \\ 0 & 1 & \frac{-y_{prp}}{z_{prp} - z_{vp}} & \frac{y_{prp}z_{vp}}{z_{prp} - z_{vp}} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{-1}{z_{prp} - z_{vp}} & \frac{z_{prp}}{z_{prp} - z_{vp}} \end{bmatrix}$$

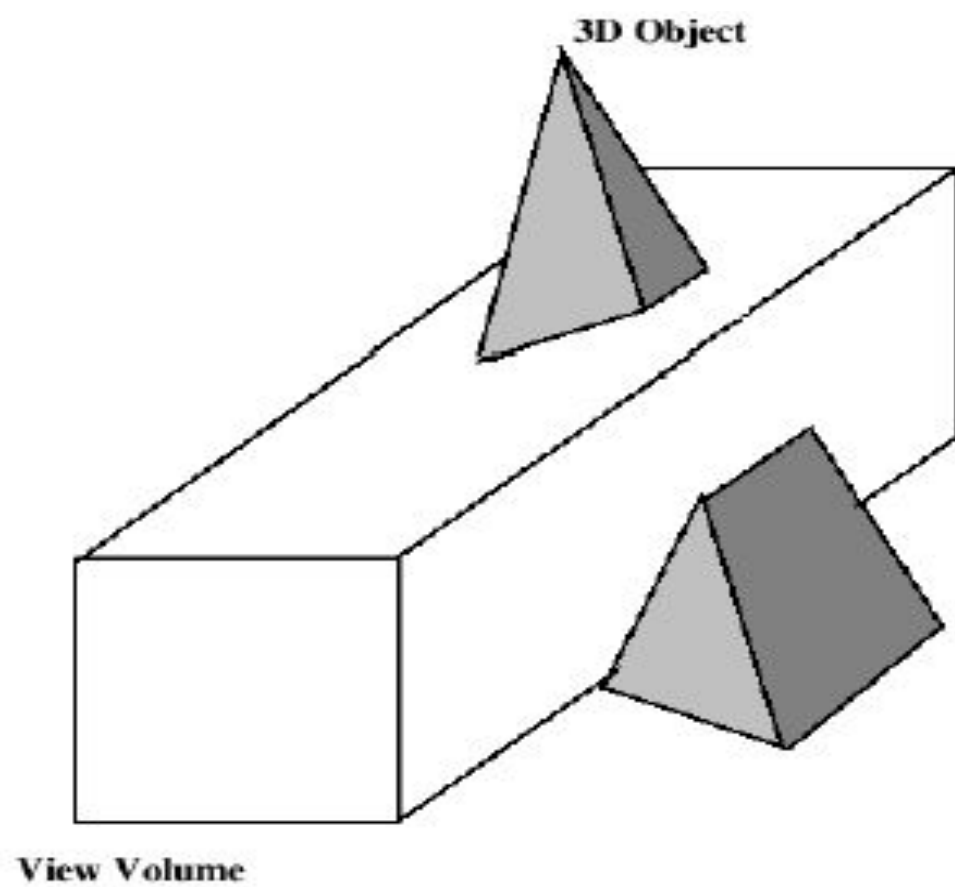
Therefore, the overall homogeneous transformation matrix of the system is

- Therefore, the general perspective-projection transformation can be expressed in matrix form as

$$\mathbf{M}_{\text{perspective}} = \mathbf{M}_{\text{scale}} \cdot \mathbf{M}_{\text{shear}}$$

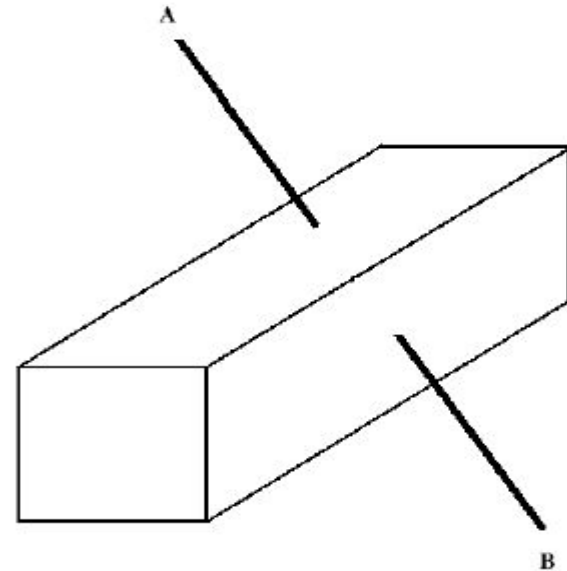
Clipping

- An algorithm for three-dimensional clipping identifies and saves all surface segments within the view volume for display on the output device.
- All parts of objects that are outside the view volume are discarded.
- Instead of clipping against straight-line window boundaries, we now clip objects against the boundary planes of the view volume.



Clipping a Line Segment

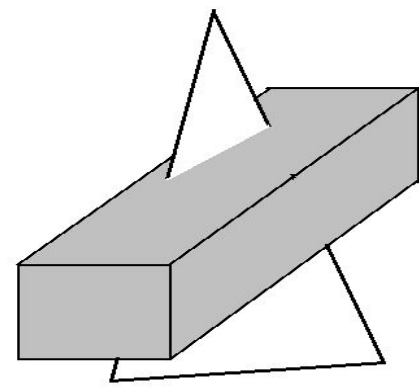
- To clip a 3D line segment, we need to test the position of the line using rectangular view volume boundary planes.



- In the above figure, the rectangular view volume has six planes.

- If we have the end points of the line (A and B), then by substituting these end points in each of the plane equations, we can determine whether the end points lie inside or outside that plane.
- Let (x_1, y_1, z_1) is one end point of the line. We have the plane equation
$$Ax + By + Cz + D = 0.$$
- Substituting (x_1, y_1, z_1) , we get
$$Ax_1 + By_1 + Cz_1 + D.$$
- If $Ax_1 + By_1 + Cz_1 + D > 0$, then the point (x_1, y_1, z_1) is outside the plane boundary. If it is less than zero, the point is inside the plane boundary.

Polygon Clipping



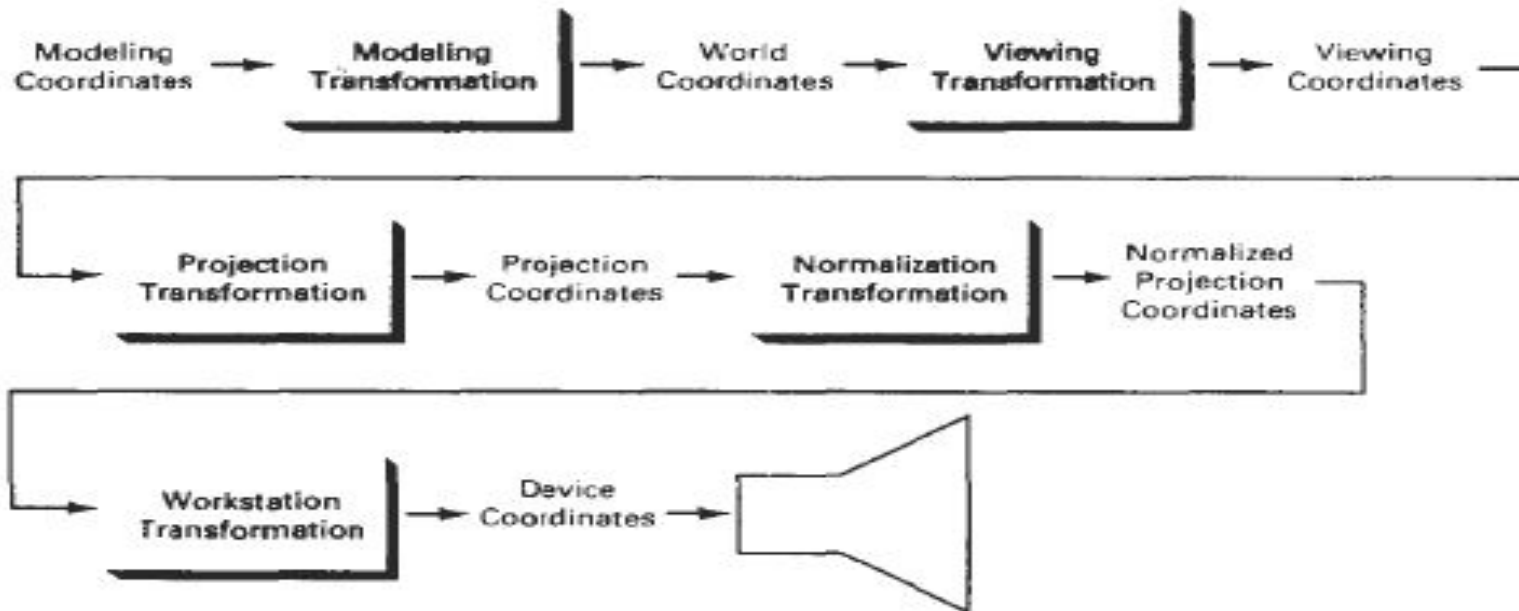
- If we have a polygon surface of a 3D object as shown in the above figure, we need to consider individual polygon edges for clipping.
- Each of these edges can be clipped against the rectangular view volume.
- First, the coordinate extents are tested against each boundary of the view volume to decide whether the object is completely inside or completely outside the boundaries.
- If the coordinate extents are inside, save those vertices.
- If the coordinate extents are outside, discard those vertices. Otherwise, intersection calculations are performed. The procedure is exactly similar to that of 2D polygon clipping.

Normalized View Volumes

- At the first step, a scene is constructed by transforming object descriptions from modeling coordinates to world coordinates.
- Next, a view mapping convert: the world descriptions to viewing coordinates. At the projection stage, the viewing coordinates are transformed to projection coordinates, which effectively converts the view volume into a rectangular parallelepiped.
- Then, the parallelepiped is mapped into the unit cube, a normalized view volume called the normalized projection coordinate system.
- The mapping to normalized projection coordinates is accomplished by transforming points within the rectangular parallelepiped into a position within a specified three-dimensional viewport, which occupies part or all of the unit **cube**.
- Finally, at the workstation stage, normalized projection coordinates are converted to device coordinates for display.

- **The** normalized view volume is a region defined by the planes

$$X=0, x=1, y=0, y=1, z=0, z=1$$



- Advantages to clipping against the unit cube instead of the original view volume or even the rectangular parallelepiped in projection coordinates.
- First, the normalized view volume provides a standard shape for representing any sized view volume.
- This separates the viewing transformations from any workstation considerations, and the unit cube then can be mapped to a workstation of any size.
- Second, clipping procedures are simplified and standardized with unit clipping planes or the viewport planes, and additional **clipping** planes can be specified within the normalized space before transforming to device coordinates.

- Third, depth cueing and visible surface determination are simplified, since the **z** axis always points toward the viewer (the projection reference point has now been transformed to the **z axis**).
- Front faces of objects are those with normal vectors having a component along the positive **z** direction; and back surfaces are facing in the negative **z** direction.

- Mapping positions within a rectangular view volume to a three-dimensional rectangular viewport is accomplished with a combination of **scaling and translation**, express the three-dimensional transformation matrix for these operations in the form

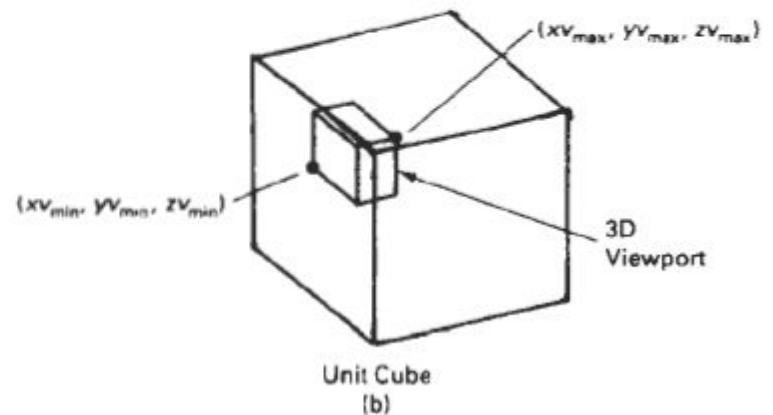
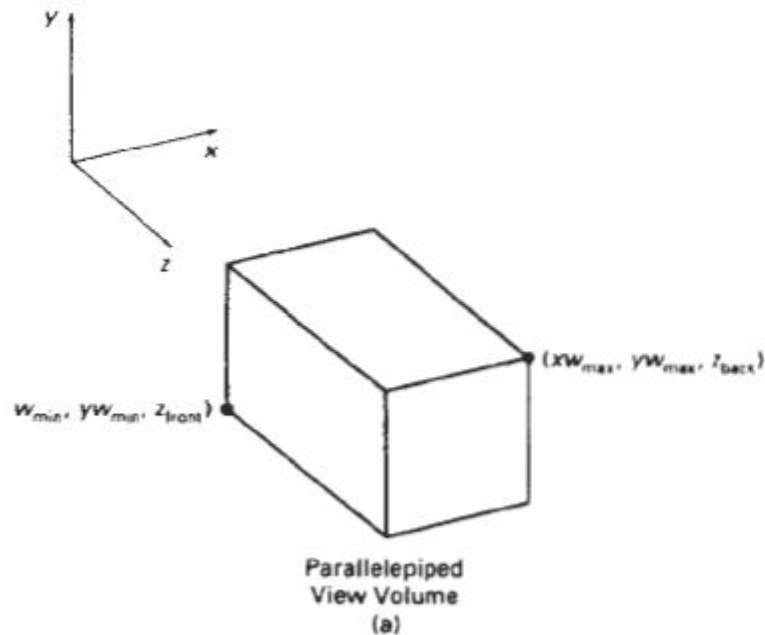
$$\begin{bmatrix} D_x & 0 & 0 & K_x \\ 0 & D_y & 0 & K_y \\ 0 & 0 & D_z & K_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Factors **D_x, D_y** , and **D_z** , are the ratios of the dimensions of the viewport and **regular** parallelepiped view volume in the **x** , **y** , and **z** directions

$$D_x = \frac{xv_{\max} - xv_{\min}}{xw_{\max} - xw_{\min}}$$

$$D_y = \frac{yv_{\max} - yv_{\min}}{yw_{\max} - yw_{\min}}$$

$$D_z = \frac{zv_{\max} - zv_{\min}}{z_{\text{back}} - z_{\text{front}}}$$



- The additive translation factors **K_x** , **K_y** , and **K_z** , in the transformation **are**

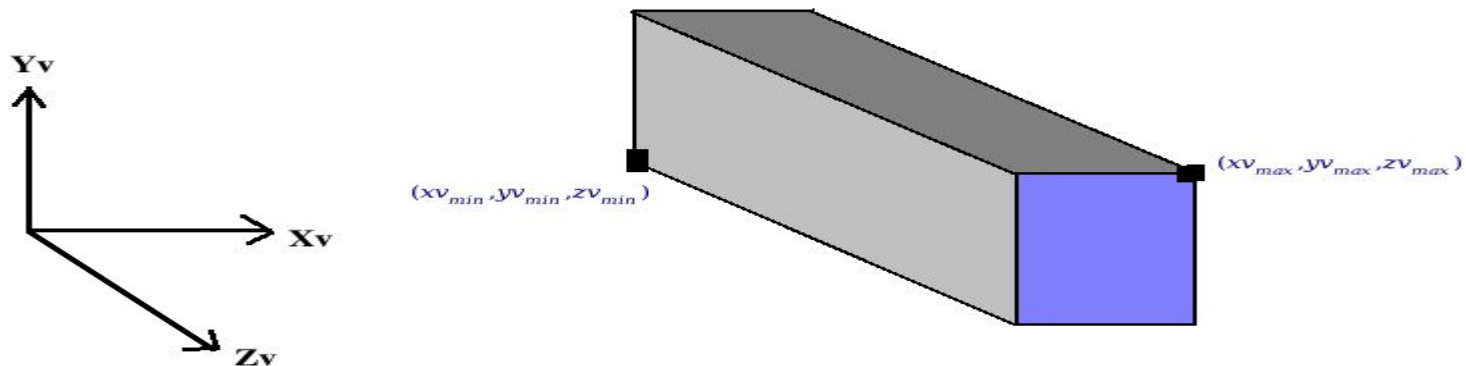
$$K_x = xv_{\min} - xw_{\min}D_x$$

$$K_y = yv_{\min} - yw_{\min}D_y$$

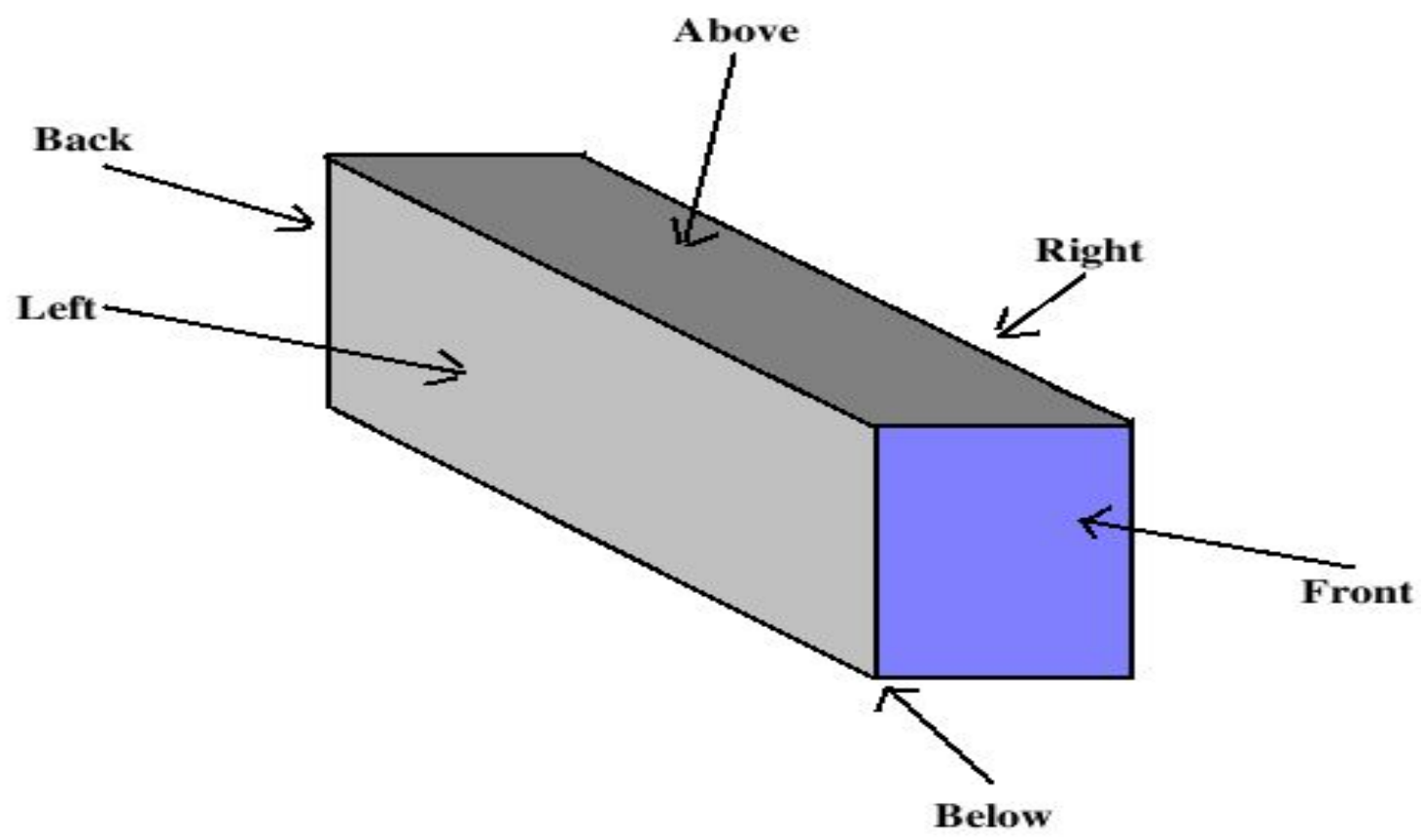
$$K_z = zv_{\min} - z_{\text{front}}D_z$$

Viewport Clipping

- For three dimensional points, we need to expand the region code to six bits. Each point in the description of a scene is then assigned a six-bit region code that identifies the relative position of the point with respect to the viewport.
- Consider the rectangular view volume shown below. Let the rectangular box is defined with the 2 points $(xv_{max}; yv_{max}; zv_{max})$ and $(xv_{min}; yv_{min}; zv_{min})$ as shown below.



- The different positions around this view volume (front, back, above, below, right, left) are shown below



- Each of the regions is assigned a six bit outcode as follows:
- Let (x, y, z) is one end point of a line, then the bits are set as,
 - Bit 1 = 1, if $x < x_{vmin}$ (Left)
 - Bit 2 = 1, if $x > x_{vmax}$ (Right)
 - Bit 3 = 1, if $y < y_{vmin}$ (Below)
 - Bit 4 = 1, if $y > y_{vmax}$ (Above)
 - Bit 5 = 1, if $z < z_{vmin}$ (Front)
 - Bit 6 = 1, if $z > z_{vmax}$ (Back)

- For example, A region code 101000 identifies a point above and back of the view volume.
- A region code 000000 identifies a point inside the view volume.
- Suppose a line segment with both end points are given.
- If the region code for both endpoints are 000000, the line is completely inside the view volume.
- If the logical AND of the region codes contain a 1 or not zero, then the line is completely outside the view volume. The line can be discarded. Otherwise partially inside.

- [If the above two conditions do not work, then a part of the line is inside and a part is outside the view volume].
- We test for intersections of the line with the surrounding planes of the view volume.
- we use the calculated intersection of a line with a viewport plane to determine how much of the line can be thrown away.
- The remaining part of the line is checked against the other planes, and continue until either the line is totally discarded or a section is found inside the **Clipping** volume.

- Let a 3D line segment with end points (x_1, y_1, z_1) and (x_2, y_2, z_2) are given.
- The equation of the line can be written in parametric form as,

$$x = x_1 + (x_2 - x_1)u, \quad 0 \leq u \leq 1$$

$$y = y_1 + (y_2 - y_1)u$$

$$z = z_1 + (z_2 - z_1)u$$

- where u varies from 0 to 1.
- where (x, y, z) is any point on the line.



- At P1, value of u is 0.
- At P2, value of u is 1.
- To find the intersection of this line with a plane of the rectangular box (view volume), substitute the coordinate value for the plane in the appropriate parametric expression and solve for u .

- Testing a line against the ***zV_{min}***, plane of the viewport.

$$u = \frac{z_{V_{min}} - z_1}{z_2 - z_1}$$

- When the calculated value for ***u*** is not in the range from 0 to 1, the line segment does not intersect the plane under consideration at any point between endpoints **P1** and **P2**
- If the calculated value for ***u*** is in the interval from **0** to 1, we calculate the intersection's ***x*** and ***y*** coordinates as

$$x_l = x_1 + (x_2 - x_1) \left(\frac{z_{v_{min}} - z_1}{z_2 - z_1} \right)$$

$$y_l = y_1 + (y_2 - y_1) \left(\frac{z_{v_{min}} - z_1}{z_2 - z_1} \right)$$

- If either **x1** or **y1** is not in the range of the boundaries of the viewport, then this line intersects the front plane beyond the boundaries of the volume

Clipping in Homogeneous Coordinates

- As each coordinate position enters the transformation pipeline, it is converted to a homogeneous-coordinate representation:

$$(x, y, z) \rightarrow (x, y, z, 1)$$

- The various transformations are applied and we obtain the final homogeneous point:

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- Where the homogeneous parameter h may not be 1. In fact, h can have any real value. Clipping is then performed in homogeneous coordinates, and clipped homogeneous positions **are** converted to nonhomogeneous coordinates in three dimensional normalized-projection coordinates:

$$x' = \frac{x_h}{h}, \quad y' = \frac{y_h}{h}, \quad z' = \frac{z_h}{h}$$

- We will, of course, have a problem if the magnitude of parameter **h** is very small or **has** the value 0; but normally this will not **occur**, if the transformations are carried out properly.
- At the final stage in the transformation pipeline, the **normalized** point is transformed to a three-dimensional device coordinate point.
- The **xy** position is plotted on the device, and the **z** component is used for depth-information processing.

- Setting up clipping procedures in homogeneous coordinates allows hardware viewing implementations to **use** a single procedure for both parallel and perspective projection transformations.
- Objects viewed with a parallel projection could be correctly clipped in three dimensional normalized coordinates, provided the value $h = 1$ has not been altered by other operations.
- But perspective projections, in general, produce a homogeneous parameter that no longer has the value 1. Converting the sheared frustum to a rectangular parallelepiped can change the value of the homogeneous parameter.

- So we must clip in homogeneous coordinates to be sure that the clipping is carried out correctly. **Also**, rational spline representations are **set** up in homogeneous coordinates with arbitrary values for the homogeneous parameter, including $h < 1$.
- Negative values for the homogeneous parameters can-also be generated **in** perspective projection.
- When coordinate positions are behind the projection reference point.
- This can occur in applications where we might want to move inside of a building or other object to view its interior.

- To determine homogeneous viewport clipping boundaries, we note that any homogeneous-coordinate position $(x_h, y_h, z_h, \mathbf{h})$ inside the viewport if it satisfies the inequalities

$$xv_{\min} \leq \frac{x_h}{h} \leq xv_{\max}, \quad yv_{\min} \leq \frac{y_h}{h} \leq yv_{\max}, \quad zv_{\min} < \frac{z_h}{h} \leq zv_{\max}$$

- Thus, the homogeneous clipping limits are

$$\begin{array}{llll}
 hxv_{\min} \leq x_h \leq hxv_{\max}, & hyv_{\min} \leq y_h \leq hyv_{\max}, & hzv_{\min} \leq z_h \leq hzv_{\max}, & \text{if } h > 0 \\
 hxv_{\max} \leq x_h \leq hxv_{\min}, & hyv_{\max} \leq y_h \leq hyv_{\min}, & hzv_{\max} \leq z_h \leq hzv_{\min}, & \text{if } h < 0
 \end{array}$$